THE COMPUTATIONAL COMPLEXITY OF TAKING SCOPE

Ned Sanger

1. Introduction

Linguistic theories usually begin from the assumption that natural languages are by and large compositional. If a sentence is composed of two parts, X and Y, its meaning should therefore be derivable from the meanings of X and Y. As reasonable as that idea sounds, it faces a number of challenges, one of which is the phenomenon of *scope displacement*. Standard constituency tests suggest, for instance, that the sentence "Alice sheared every sheep" is syntactically composed of a subject "Alice" and a predicate "sheared every sheep." But it's also widely accepted that the sentence's meaning is computed from the meaning of the direct object "every sheep" and its context "Alice sheared __." It therefore looks like the grammatical structure of even this simple sentence does not align with the logical structure of the thought it conveys.

In the face of this apparent mismatch, one possible way of maintaining compositionality is to revise one's syntactic theory so that the sentence really does decompose into the meaningful parts that the semantics suggests. Another possibility is to revise one's semantic theory so that the meaning of the sentence becomes computable from the meaning of its syntactic parts. But the most prominent approach in generative linguistics is to take the mismatch at face value. The solution, it's thought, is to specify rules that systematically transform the surface grammatical structure of a sentence into its logical structure. On this approach, interpretation begins by applying a syntactic operation called *Quantifier Raising* (QR) that moves expressions from the positions where they are spoken into the positions where they take scope. Once QR has applied as many times as needed, one is left with a syntactic object called a *Logical Form* (LF). It is this LF, rather than the surface structure, that provides the actual input to the compositional semantics. Applied to the sentence above, QR rearranges the surface constituency "[Alice] [sheared every sheep]" into the logical constituency "[every sheep] [Alice sheared __]."

Although this framework has been widely adopted in semantics research, some of the most basic questions about its foundations remain unresolved. In particular, it's not known whether the mapping from surface structure to LF is, in general, efficiently computable. In practice it is, to be sure, usually easy to find at least one well-formed LF for a sentence, but that isn't always the case. Sometimes an LF can differ surprisingly from the surface structure of a sentence and require some cleverness to construct. Because a sentence can be related to multiple, truth-conditionally distinct LFs, it can also be hard to know whether one has found all of its possible LFs or only some of them.

There is therefore both a practical and a theoretical reason to study the foundations of scope-taking operations like Quantifier Raising. The practical reason is that it would be useful to know whether there are efficient algorithms which automate the semanticist's task of building LFs out of surface forms. The theoretical reason is that we should understand what kind of computational mechanisms are necessary in order to use a grammar that allows for scope-taking. If the search space of possible LFs for a sentence were efficiently explorable, that would be an important fact about grammar. If it were not, then

a key question would arise: what properties of the LFs that semanticists work with make them easy (for the most part) to find?

The main result of this paper is that the search space is *not* efficiently explorable. More precisely, the following problem is computationally intractable (NP-complete): given a syntactic structure containing expressions that are semantically uninterpretable in their surface positions, is it possible to build a semantically interpretable LF from it by repeatedly applying QR? Because there is no efficient and general algorithm for finding even *one* LF for a sentence, there is a fortiori no efficient way of finding *all* of them. These mathematical facts reveal a thorny computational problem at the center of some of the most widely used semantic frameworks.

The structure of this paper is as follows. Section 2 presents some empirical data, in order to demonstrate the need for scope-taking operations and to introduce the facts they must account for. Section 3 then describes Quantifier Raising and a couple of other mechanisms for taking scope, and argues that all of them give rise to a nontrivial computational decision problem. Section 4 mathematically formalizes Logical Forms, Quantifier Raising, and the computational problem that the remainder of the paper studies. Section 5 and Section 6 prove that this problem is NP-complete. Section 7 concludes.

2. The Problem of Scope

Scope-taking expressions pose a challenge to compositionality because they occur syntactically embedded inside of the expressions that provide their semantic arguments. Section 1 illustrated this using the example "Alice sheared every sheep." This sentence's truth value comes from applying the generalized quantifier *every sheep* to the predicate P that is true of an individual x iff Alice sheared x. One immediate problem is that no constituent of the sentence denotes P. The surface syntax therefore does not straightforwardly provide the pieces necessary to assemble the desired meaning.

There is another equally pressing problem: since the verb phrase "sheared every sheep" *is* a constituent, we expect it to have some meaning built out of its two parts, the verb *sheared* and the object *every sheep*. But what could it be? A standard assumption is that a transitive verb like *sheared* denotes the binary relation that holds of x and y iff y sheared x. And since Montague (1973), semanticists have assigned a higher-order meaning to expressions like *every sheep*: specifically, they denote generalized quantifiers, functions from sets of individuals to truth values. Unfortunately, the two standard modes of semantic combination, Function Application and Predicate Modification (Heim and Kratzer 1998), cannot combine a generalized quantifier with a binary relation between individuals. The syntax therefore insists on assembling a meaning out of components which the semantics does not know how to fit together. Formally, the problem that we face is a *type clash*: it simply is not possible with the standard toolkit to combine something of type $(e \to t) \to t$ with something of type $e \to (e \to t)$.

A third challenge comes from sentences that contain more than one scope-taking element:

(1) Exactly three tenors performed in every opera.

Sentence (1) would be true in any situation where there are exactly three tenors who performed in every opera. It would also be true in any situation where every opera is such that exactly three tenors performed in it. Because either of these two types of situation could be the case without the other being the case, the sentence is ambiguous. This particular ambiguity has to do with scope, since (thinking in procedural terms) the

difference in meaning comes down to the following: in what order are the meanings of *exactly three tenors* and of *every opera* integrated during the composition of meaning? Do we first find three tenors, and then verify that for every opera, each of them performed in it? This scopal ordering is the right one for the first situation. Or do we instead verify that for every opera, we can find three tenors who performed in it? This scopal ordering is right for the second situation. An empirically adequate compositional semantics should allow for both possibilities. More precisely, it should treat each of the following formulas of higher-order logic as a possible meaning of (1):

(2) a. $\textsc{exactly-three-tenors}(\lambda x.\textsc{every-opera}(\lambda y.\textsc{performed-in}\ y\,x))$
b. $\textsc{every-opera}(\lambda y.\textsc{exactly-three-tenors}(\lambda x.\textsc{performed-in}\ y\,x))$

Now consider what happens when we replace the upward-monotonic quantifier *every opera* with the downward-monotonic quantifier *none of the operas*:

(3) Exactly three tenors performed in none of the operas.

Unlike (1), this sentence is unambiguous. It's true in a situation where there are three tenors who did not perform in any opera. But it wouldn't be made true by a situation where none of the operas featured three tenors. That is, the sentence can mean (4a) but not (4b).

(4) a. $\textsc{exactly-three-tenors}(\lambda x.\textsc{no-opera}(\lambda y.\textsc{performed-in}\ y\,x))$
b. $\textsc{no-opera}(\lambda x.\textsc{exactly-three-tenors}(\lambda y.\textsc{performed-in}\ y\,x))$

The compositional challenge here is that (1) and (3) have the same coarse syntactic structure, as shown by Figure 1. Semantic interpretation must therefore associate two meanings with the structure on the left, but only one meaning with the similar-looking structure on the right. This is the problem of *non-uniformity*: not all quantificational phrases enjoy the same scope-taking possibilities (Szabolcsi 1997; 2010).

To recap, there are three problems that a theory of scope needs to solve. The first is that there is often no constituent which denotes the appropriate argument for a scope-taking expression. The theory of scope must nonetheless construct this argument and pass it to the scope-taker. The second problem is that scope-takers have higher-order meanings that seem to create type clashes when interpreted in situ. The theory must include a mechanism that resolves this clash. The third problem, which is more fine-grained and empirical, is to derive scope ambiguities when they arise, and to explain why they do not arise when they don't. The first two problems, whose solution will require a general mechanism for compositionally interpreting natural language quantifiers, are the central ones for this paper. But the remainder of this section, whose purpose is to provide some empirical orientation, will consider the third issue too.

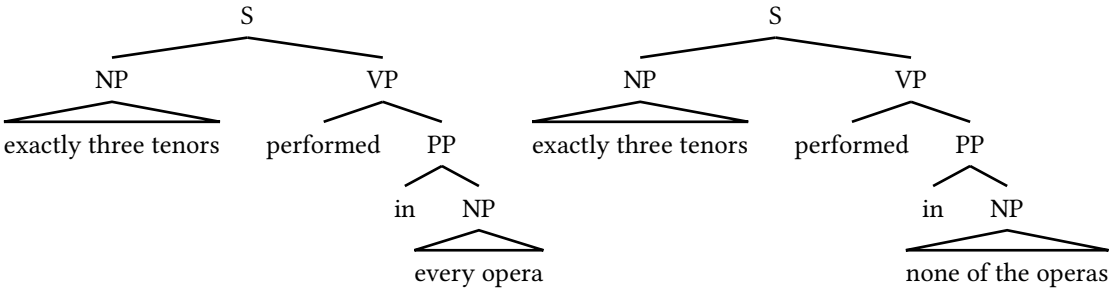


Figure 1: Surface syntax of sentences (1) and (3).

When presenting the problems associated with scope-taking, it's common practice to focus on sentences with two interacting quantificational noun phrases, just like I've done in (1) and (3). This focus on just one type of example may give the misleading impression that these issues are peculiar to the semantics of noun phrases. To show that the problems actually crop up in a wide band of semantically interesting constructions, the rest of this section will therefore describe other parts of English grammar to which they are relevant.

A first case involves *modals* and *negation* (Iatridou and Zeijlstra 2013; Kubota and Levine 2019). Consider the sentences in (5), each of which combines negation with an epistemic or deontic modal. The annotations on the right indicate whether the modal is epistemic or deontic, as well as its scope with respect to negation.

(5) a. Alice might not leave. $\Diamond_e \succ \neg$
b. Alice must not leave. $\Box_d \succ \neg$
c. Alice may not leave. $\Diamond_d \prec\!\succ \neg$
d. Alice need not leave. $\neg \succ \Box_d$

In example (5a), the existential epistemic modal *might* scopes obligatorily above *not*. If it were possible to scope under negation, then the sentence could mean that there is no chance of Alice leaving; this stronger interpretation is unavailable. In (5b), the universal deontic modal *must* has to scope above *not*. In (5c), the existential deontic modal *may* can scope either above or below *not*. (To bring out the reading where *may* scopes above negation, consider a sentence like this: "You may not eat your broccoli, but then you must eat your peas instead.") In (5d), the universal deontic modal *need* obligatorily scopes under *not*.

This small dataset raises questions similar to the ones we saw earlier. What explains the scope ambiguity in (5c)? Why do different modals show different behavior with respect to negation? If we moreover make the reasonable assumption that modal operators and negation each require a propositional argument, then all four sentences in (5) also contain a surface type clash. Just like earlier, a theory of scope will need to say how to resolve it.

Another source of interesting scope-takers is *comparatives*, which can scope both over and under modals (Stateva 1999). The example below follows Heim (2000). Imagine a journalist who, after showing a five-page draft to an editor, is told the following:

(6) The article was required to be less long than this.

The editor's statement is ambiguous. Under one interpretation, it means that the article is not permitted to be longer than five pages ("… so go make some cuts!"). Under the other interpretation, it means that the article is longer than the minimum required length ("… you've worked harder than you had to!").

Figure 2 shows how to analyze this ambiguity in terms of scope. The ingredients of the analysis are as follows: *d* is the type of degrees, *long* is a binary relation of type $d \to e \to t$, *less … than this* is a generalized quantifier of type $(d \to t) \to t$, and *required* denotes the usual universal quantifier over possible worlds. The GQ *less … than this* contains all sets of degrees that are bounded from above by five. As the Logical Forms should make clear, the only difference between the two readings is whether one quantifies first over degrees, then over worlds, or the other way around. Said differently, the ambiguity comes down to whether the meaning of the comparative is incorporated before the meaning of the modal or not.

Oddly enough, even though comparative operators create scope ambiguities when they co-occur with modals, they don't when they co-occur with quantificational noun phrases. This is the content of *Kennedy's Generalization*, which states that degree operators always

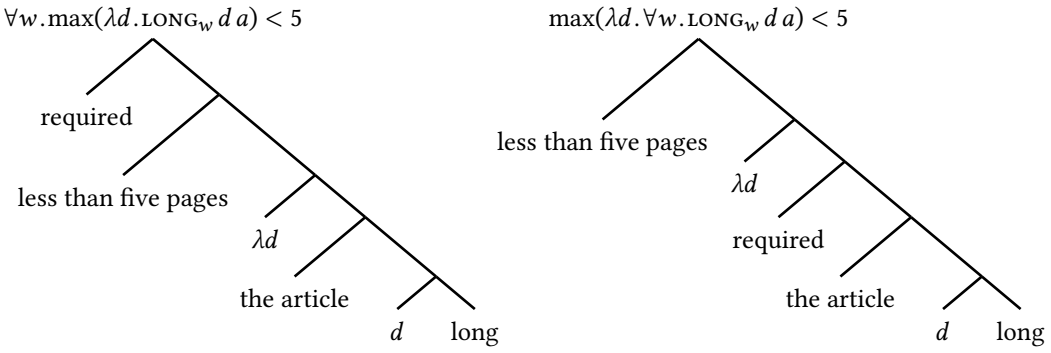


Figure 2: Logical Forms for (6).

take narrow scope with respect to quantificational noun phrases (Kennedy 1997). Here is an illustration, once again following Heim (2000). Suppose that a teacher is having a conversation about the height of his students, somebody mentions that Doug is 5'6" inches tall, and the teacher responds with (7):

(7) Every girl is less tall than that.

If it were possible for the comparative expression (*one inch ... -er than that*) to outscope *every girl*, then (7) could mean the following:

$$\max(\lambda d.\forall g.\textsc{girl}\, g \Rightarrow \textsc{tall}\, d\, g) < 5'6".$$

But this formula says only that the greatest height that every girl exceeds is under 5'6", which is itself just a long way of saying that the *shortest* girl is less than 5'6" tall. That is not, however, a possible interpretation of (7). The sentence is unambiguous, and it's the opposite scopal order that delivers correct truth conditions:

$$\forall g.\textsc{girl}\, g \Rightarrow \max(\lambda d.\textsc{tall}\, d\, g) < 5'6".$$

Since comparatives can scope both over and under modals, it's no surprise that *superlatives* can as well. Take the following sentence:

(8) Alice wants to read the longest book.

Heim (1999) points out the amazing fact that this sentence is ambiguous between *five* readings, all of which can be captured just by allowing three expressions—the modal verb, the superlative morpheme *-est*, and the direct object—to take scope in different places. To give a sense of the possibilities, here are three of those readings, along with their representations in an intensional higher-order logic. In these representations, the morpheme *-est* denotes a function of type $(\mathsf{d} \to \mathsf{e} \to \mathsf{t}) \to \mathsf{e} \to \mathsf{t}$.

(9) **Absolute *de dicto***
$\Box\exists b.\textsc{book}\, b \wedge \textsc{est}(\textsc{long})(b) \wedge \textsc{read}(b)(\textsc{ann})$
Paraphrase: Alice wants to read the longest book that exists, whatever it might be.

(10) **Relative *de re***
$\textsc{est}(\lambda d\lambda x.\exists b.\textsc{book}\, b \wedge \textsc{long}\, d\, b \wedge \Box\textsc{read}\, b\, x)(\textsc{ann})$
Paraphrase: Alice wants to read a particular book b, and all of the books that other people want to read are shorter than b.

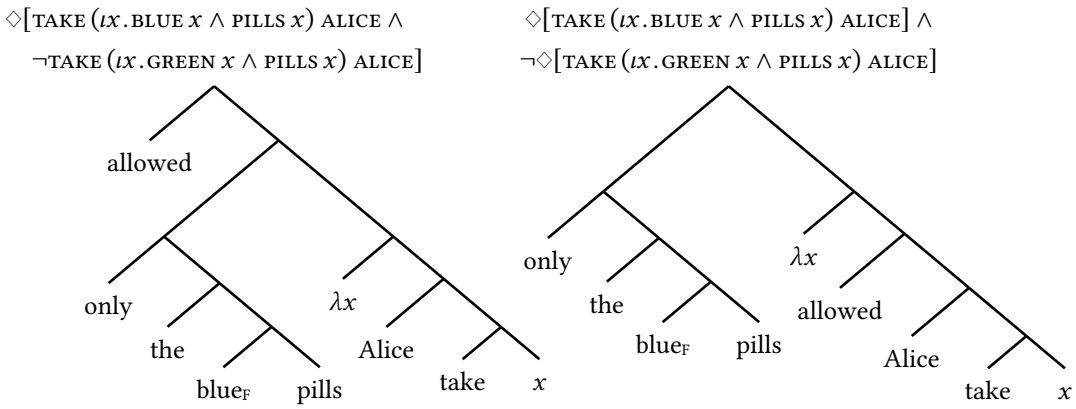


Figure 3: Logical Forms for (12).

(11) **Upstairs *de dicto***
EST($\lambda d \lambda x . \Box \exists b . \text{BOOK}\, b \wedge \text{LONG}\, d\, b \wedge \text{READ}\, b\, x$)(ANN)
Paraphrase: Alice wants to read a certain length of book that exceeds what anybody else wants to read.

This example shows that even in short sentences, scope-takers can stack up quickly, resulting in a roughly factorial explosion of ambiguity. It also shows that finding all the possible readings of a sentence—and even just finding all the well-typed arrangements of its scope-takers—is not trivial.

My final example of a scope-taker involves *focus-sensitive operators* like the word *only*. As McCawley (1996) and others point out, sentences like (12) are ambiguous.

(12) Alice is allowed to take only the *blue* pills.

Imagine there are only two relevant pill colors: blue and green. Then under one interpretation, this sentence means: Alice is allowed to only take the blue pills (but she may take the green ones too, if she likes). Under another interpretation, it means: Alice is only allowed to take the blue pills (the green ones are forbidden). As these paraphrases already suggest, the difference between the two readings is about the scope of *only*: do we interpret it inside or outside of the modal *allowed*? Inside leads to the first interpretation, outside to the second. The LFs in Figure 3 show these two possibilities. Notice that the word *only* combines with two arguments. The first is an expression ("the *blue* pills") containing a focused subexpression ("*blue*"), the one that *only* will contrast with some set of alternatives. The second argument is an expression that denotes a proposition. Since the surface position of *only* is embedded inside the proposition that it ultimately needs to take as its second argument, there is a surface type clash that must be resolved by taking scope.

That concludes the quick tour of scope-taking expressions. Here are the takeaways. Scope-taking expressions, and the concomitant challenge of interpreting them when they show up embedded inside their arguments, are pervasive. We find them in the semantics of quantificational noun phrases, of modality, of degrees and comparison, of focus, and beyond. A compositional semantics will need to meet this challenge by providing a way to take scope: a controlled mechanism for dissociating the order of syntactic and semantic composition.

We also saw that this mechanism needs to have some flexibility built into it. The reason is that higher-order meanings can, at least some of the time, take scope in more

than one place, with different choices leading to the different readings of a sentence. But the mechanism also needs to be constrained somehow, because not every well-typed arrangement of scope takers leads to a possible reading of the sentence under study. The rest of this paper prioritizes scope-taking mechanisms that maximize flexibility, for two reasons: (i) they are the most popular mechanisms in the literature, and (ii) they are the ones that lead to computationally interesting decision problems.

The next section presents some candidates for what that mechanism might be.

3. Some Ways to Take Scope

Semanticists have invented and defended a range of methods for resolving type-clashes and deriving scope ambiguities, including:

- Quantifier Raising (May 1977; Heim and Kratzer 1998),
- Cooper Storage (Cooper 1983; Kobele 2018),
- Continuations (Barker and Shan 2014; Kiselyov and Shan 2014),
- Quantifying In (Montague 1973),
- Flexible Montague Grammar (Hendriks 1993).

This section introduces the first three techniques by showing how they derive the inverse scope reading of the sentence *a critic panned every film*, and then justifies the choice to ultimately focus on QR. It's not the intention to comprehensively introduce these scope-taking operations; see Barker (2015) for a more detailed exposition along those lines. Instead the purpose is to (re)familiarize the reader with these three ways of resolving type clashes, and show that each of them is associated with a computational decision problem of the following shape: given an ill-typed input structure, can repeated use of the scope-taking operation make it well-typed?

Readers who just want to get to the mathematical results of this paper right away can skim Section 3.1 and then skip to Section 4. Nothing later on will depend on the contents of Section 3.2 and Section 3.3.

3.1. Quantifier Raising

The idea of QR is to change the syntactic structure that gets semantically interpreted by displacing expressions into positions that directly reflect where they take scope. Each use of QR has two steps. First, the element that takes scope is raised, and a variable is placed in its original position. Second, a lambda abstraction that binds this variable is placed directly below the expression's new scopal position. Figure 4 shows how two applications of QR can derive the linear scope reading of the sentence *a critic panned every film*. Notice that the initial structure contains a type clash: the semantic rules in Heim and Kratzer (1998) can't combine *panned*, which has type $\mathsf{e} \to \mathsf{e} \to \mathsf{t}$, with *every film*, which has type $(\mathsf{e} \to \mathsf{t}) \to \mathsf{t}$. This problem is resolved in the post-QR structure, which can be interpreted using nothing but their rules of Function Application and Predicate Abstraction. The meaning assigned to the entire tree is

$$\textsc{some}(\textsc{critic})(\lambda y^{\mathsf{e}}.\textsc{every}(\textsc{film})(\lambda x^{\mathsf{e}}.\textsc{panned}\, x\, y))).$$

If QR had raised *a critic* to a position below *every film* (or, in this case, not applied to *a critic* at all), then the result would be the inverse scope reading:

$$\textsc{every}(\textsc{film})(\lambda x^{\mathsf{e}}.\textsc{some}(\textsc{critic})(\lambda y^{\mathsf{e}}.\textsc{panned}\, x\, y)).$$

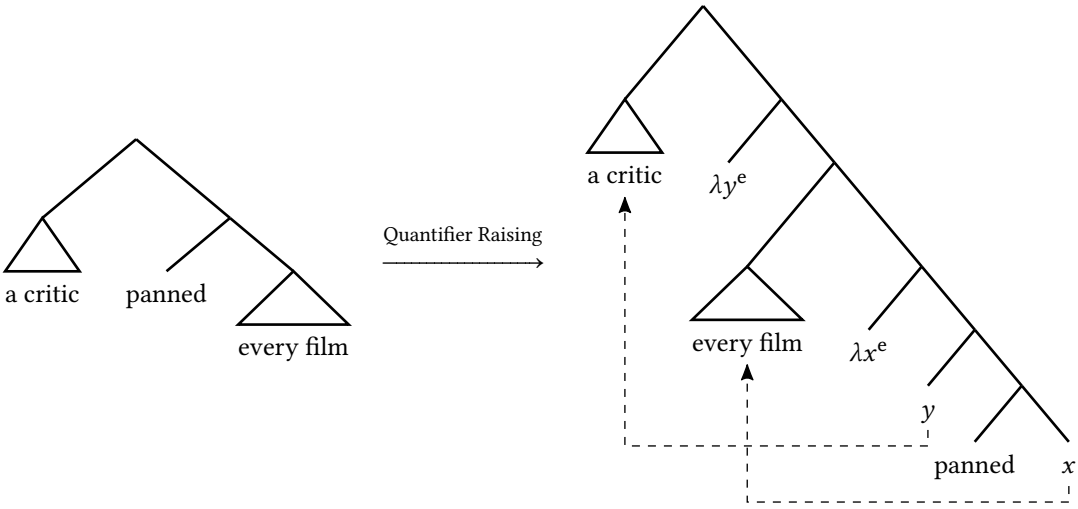


Figure 4: Quantifier Raising in action.

A variable left behind by QR must be assigned a type. The one in Figure 4 has type e, which I've indicated with a superscript on the λ-abstraction that binds it, but in most presentations of QR, any type that leads to an interpretable result is allowed. This flexibility significantly increases the number of type clashes that QR can resolve. Figure 2 showed, for example, the utility of leaving behind degree traces (type d) to analyze comparatives. Also useful, but more controversial, are *higher-order traces*. These were first used when studying the interpretation of *wh*-movement rather than QR. Rullmann (1995), for example, uses traces of type $s \to (s \to e \to t) \to t$ to derive the reading of (13) where *books* takes narrow scope with respect to *need*.

(13) How many books does John need to read?

Under the relevant interpretation, there might not be any specific books that John needs to read; he just needs to read a specific number, and the question asks what that number is. That said, higher-order traces have also been used with QR, for example to analyze certain scope-reconstruction effects in Hindi (Poole and Keine 2024). And Charlow (2021) shows that traces with the type of a dynamic generalized quantifier, $((g \to e) \to g \to g \to t) \to g \to g \to t$, where g is the type of assignment functions, can derive some elusive cumulative readings of sentences with modified numerals. Although there are some empirical arguments for restricting traces to atomic types (Landman 2006; Poole 2024), this constraint turn outs to have no effect on the computational tractability of QR and I will therefore ignore it. See Section 7 for some more discussion.

Traditionally, QR has been understood as a *covert* version of the movement operations used in transformational theories of syntax, which is to say, it displaces an expression for the purposes of semantic interpretation, but—unlike, say, *wh*-movement in English —has no effect on pronunciation. This idea is especially natural in an "inverted Y" model of grammar (Figure 5), where sentences are associated with more than one level of syntactic representation, and covert movement is just the name for movement that happens somewhere between Surface Structure and Logical Form. So historically, use of QR has gone hand-in-hand with the assumption that part of understanding a sentence is the construction of a syntactic structure that explicitly represents everything needed for semantic interpretation: relative quantifier scope, the scope of modals and negation,

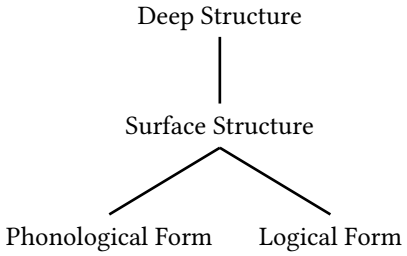


Figure 5: The inverted Y model of Government and Binding Theory.

binding, referential opacity, association with focus, existential closure, etc. But a level of Logical Form is not strictly necessary. QR can be used just as well in a theory where sentences are associated with only one syntactic representation, so long as that representation is rich and the mappings to form and to meaning are flexible, like in the "single-output syntax" in Bobaljik (2002).

At any rate, whatever background theory one adopts, the appeal and the promise of QR have always been the same: that constraints on semantic scope could reduce, at least in part, to constraints on syntactic movement. Semanticists are not, I think, as sanguine about this prospect as they once were. On the one hand, indefinites and focused expressions will happily scope outside any number of syntactic islands; on the other hand, distributive universals like *every* struggle mightily to scope out of tensed clauses that *wh*-movement easily escapes. It seems unlikely that syntactic islands and scope islands could be identified.

As a result, QR seems to get used in two ways in the literature. Sometimes, to be sure, it retains its traditional formulation as a covert movement operation that targets mainly noun phrases and resolves clashes between a quantifier's type and its environment. But at other times, especially in purely semantic discussions, QR looks more like a convenient way to get the effects of scope-taking while staying noncommittal about the underlying mechanism of it. If it's clear that an expression must take scope over some portion of its context, then one might simply say it "QRs" to that position, with no regard for whether the displacement resembles overt movement or not. The second understanding of QR, according to which it's just a no-frills statement of what it means to take scope which could be fleshed out in any number of ways, is the one relevant to this paper.

If we understand QR in this way, then the only non-negotiable constraint on it is that its final output be semantically interpretable, that is, well-typed. This constraint leads immediately to the central computational question associated with QR. Section 4.2 defines it formally as a decision problem called LF REPAIR, but for now it's enough to state it informally: given a syntactic structure, can repeated uses of QR make it well-typed? To answer the question is to say what expressions raise, what positions they raise to, and what trace types they leave behind. Section 6 proves that this problem is NP-complete. In the context of the inverted Y model, this fact provides a rough characterization of the computational complexity of the branch leading from Surface Structure to Logical Form.

3.2. Cooper Storage

Cooper Storage is a semantic way of resolving type clashes, designed with the goal of interpreting quantificational expressions while modifying the syntactic derivation as little as possible. Here is Cooper's own description of his idea: "if we think of … the

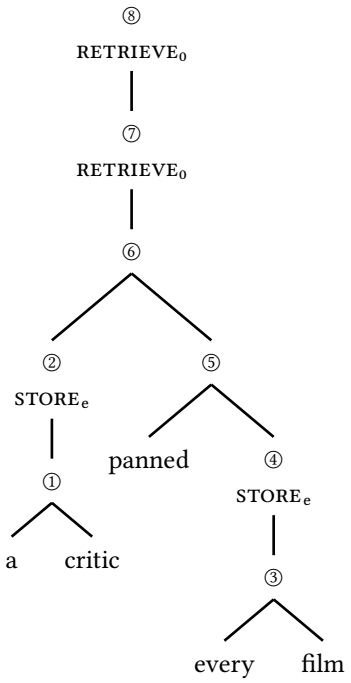


Figure 6: Cooper Storage.

semantics as working from the bottom of the tree (where the words are) to the top, the storage technique involves putting an NP interpretation on ice for a while until you have interpreted enough of the tree to represent the scope you want to give the NP. At that point you may take the NP-interpretation out of storage and quantify it in" (Cooper 1983, 55). It's worth pointing out that this technique allows expressions of any category to take scope, even though Cooper only mentioned NPs.

To implement this idea, he upgrades the standard semantic interpretation function so that the meaning assigned to each subtree of a syntactic structure is a pair of objects. The first component of that pair is called the *main expression*. Ignoring a complication that I address in the next paragraph, we can think of it as containing the meaning that a traditional interpretation function would assign to that subtree. The second component is the *store*: a list of meanings that are waiting for their scopes to be constructed, after which they can be removed from the store and integrated into the main expression.

The only complication arises when a subtree contains expressions that need to take scope. Then the main expression will be interpreted as if it contained a variable in the place of each scope-taker. This variable plays the same role as the one that a scope taker leaves behind when it undergoes Quantifier Raising. Kobele (2018) shows that it's beneficial to bind these variables immediately by adding a prefix of λ-abstractions to the main expression, and I do the same below.

The tree in Figure 6 shows how to derive the linear scope reading of *a critic panned every film* with Cooper Storage. The nodes labeled STORE$_e$ have two effects: they move a quantifier from the main expression into the store, and they introduce a variable of type e into the main expression. The nodes labeled RETRIEVE$_0$ and RETRIEVE$_1$ remove a quantifier from the store and apply its meaning to the main expression; the subscripts indicate what position the quantifier occupies inside the store.

	Main Expression	*Store*
①	$\textsc{some}(\textsc{critic})$	$\langle\,\rangle$
②	$\lambda y^{\mathrm{e}}.y$	$\langle \textsc{some}(\textsc{critic})\rangle$
③	$\textsc{every}(\textsc{film})$	$\langle\,\rangle$
④	$\lambda x^{\mathrm{e}}.x$	$\langle \textsc{every}(\textsc{film})\rangle$
⑤	$\lambda x^{\mathrm{e}}.\textsc{panned}\,x$	$\langle \textsc{every}(\textsc{film})\rangle$
⑥	$\lambda x^{\mathrm{e}}\lambda y^{\mathrm{e}}.\textsc{panned}\,x\,y$	$\langle \textsc{every}(\textsc{film}), \textsc{some}(\textsc{critic})\rangle$
⑦	$\lambda y^{\mathrm{e}}.\textsc{every}(\textsc{film})(\lambda x^{\mathrm{e}}.\textsc{panned}\,x\,y)$	$\langle \textsc{some}(\textsc{critic})\rangle$
⑧	$\textsc{some}(\textsc{critic})(\lambda y^{\mathrm{e}}.\textsc{every}(\textsc{film})(\lambda x^{\mathrm{e}}.\textsc{panned}\,x\,y))$	$\langle\,\rangle$

Table 1: Denotations of the interior nodes in Figure 6.

To make it easier to talk about this derivation, the interior nodes in Figure 6 are numbered, and Table 1 lists the contents of the main expression and the store at each node. At node ①, the main expression contains the traditional semantic value of the subtree it picks out, viz. the generalized quantifier $\textsc{some}(\textsc{critic})$, and the store is empty. At node ②, the contents of the main expression are put into storage and replaced by a variable y of type e. A λ-abstraction also binds this variable, but in Cooper's original proposal it would have been left free. The denotations at nodes ③ and ④ are computed in the same way.

At node ⑤, we have to combine the denotations of two subtrees. We do this piecewise. The new store is just the result of concatenating the stores of the two subtrees; since the store associated with the left subtree is empty, the result in this case is that the store of the right subtree persists unchanged. To obtain a new main expression, we apply the meaning of the one subtree's main expression to the other's, but with a slight catch: we do the application "under" the λ-abstraction that binds the variable y. Formally speaking, that means we combine the two main expressions, $\textsc{panned}$ and $\lambda x^{\mathrm{e}}.x$, with function composition rather than function application.

The procedure at node ⑥ is similar. The new store is again the concatenation of the two subtrees' stores. The new main expression is the result of doing function application underneath the two prefixes of λ-abstractions. The combinator that we need to combine the two expressions this time is not function composition, but

$$\lambda M\lambda N\lambda x\lambda y.Mx(Ny).$$

If we apply this to our two main expressions, $\lambda x^{\mathrm{e}}.\textsc{panned}\,x$ and $\lambda y^{\mathrm{e}}.y$, we get $\lambda x^{\mathrm{e}}\lambda y^{\mathrm{e}}.\textsc{panned}\,x\,y$.

At node ⑦, we take the meaning at index o out of the store, viz. $\textsc{every}(\textsc{film})$, and combine it with its scope. Again we have to be mindful of the prefix of binders. $\textsc{every}(\textsc{film})$ has to abstract over the same variable that was introduced when it was put into storage, namely x, so the combinator that that will join it with the main expression correctly is

$$\lambda M\lambda N\lambda y.M(\lambda x.N.y).$$

At node ⑧ we retrieve the meaning at index o, which is now $\textsc{some}(\textsc{critic})$, and combine it with the main expression using function application. At this point, the store is empty, the main expression contains the sentence's meaning, and the derivation is concluded.[1]

1. The same combinators used at nodes ⑤ and ⑥ are also used in variable-free semantics (Jacobson 1999). In that framework, function application "under the λ," so to speak, is necessary whenever something

The point of this example was just to give the reader a feel for Cooper Storage, not to present the technique in detail. It might look, therefore, like the combinators that happened to be needed at each step were conjured out of thin air. But Kobele (2018) shows how to arrive at them systematically and make rigorous the idea of "application under the λ" using the concept of a graded applicative functor, as well as how to implement different variants of the Cooper Storage technique in simply-typed and polymorphic lambda calculi.

How does Cooper's idea compare to Quantifier Raising? Ignoring the noise of the STORE and RETRIEVE nodes in Figure 6, the tree that we interpreted simply was the surface syntactic structure of the sentence. The addition of a few unary-branching nodes has no effect on the structural relationships that are ordinarily relevant for syntax, so the technique succeeds at its goal: to resolve type clashes while taking surface constituency seriously. By contrast, the entire idea of Quantifier Raising is that surface constituency is misleading and should be amended before interpretation begins. Framed in those terms, the two techniques are substantially different, and Cooper Storage can claim, for whatever it's worth, to be the more compositional.

But there is also a practical perspective from which Cooper Storage and QR are just variations on the same theme. The previous subsection said that giving a QR analysis comes down to answering three questions: what expressions raise, what positions do they raise to, and what trace types do they leave behind? The current subsection shows that giving a Cooper Storage analysis comes down to answering three exactly analogous questions: what expressions get put into the store, when do they come out of the store, and what types of variable do they introduce? It's intuitively clear that answers to either set of questions will produce corresponding answers to the other, meaning that there is a clean correspondence between QR analyses and Cooper Storage derivations. With that in mind, Cooper Storage could just as well be thought of not as a competitor to QR but as a compositional implementation of it.

3.3. Continuations

Continuations are another way of resolving type-clashes without rearranging the surface syntax. The technique was first developed to elucidate the denotational semantics of control structures in programming languages (Strachey and Wadsworth 1974). But it was later realized that the same ideas could also be used to simulate imperative control structures in functional programming languages, and then later still that they could be used to assign a compositional semantics to natural language fragments.

The basic idea is simple. Definitionally, scope takers are expressions that take (part of) their context as an argument. The challenge with interpreting in-situ quantifiers is simply that a traditional semantics doesn't provide expressions access to their context. We therefore enrich the semantics by parametrizing the interpretation function $[\![-]\!]$ with an extra argument k, called a *continuation*, which represents the meaning of the context in which an expression occurs. Expressions that need to manipulate their context may then access it through this new parameter. This strategy is similar to how, for example, Heim and Kratzer (1998) parametrize the interpretation function with an assignment function that provides context-sensitive expressions like pronouns access to the discourse

combines with an expression containing an unbound pronoun. The reason for this similarity is that Cooper Storage (as presented here) analyzes a variable waiting for a binder in the same way that Jacobson analyzes a pronoun waiting for an antecedent.

context. In that framework, the meaning of a sentence is no longer a truth value but a property of assignment functions, i.e., a function from assignments to truth values. In a continuation framework, the meaning of an expression instead becomes a property of semantic contexts.

For example, a quantifier like *every film* denotes the property of contexts that holds of k iff $k(x) = \top$ for every film x. The quantifier *a critic* denotes the property that holds of k iff $k(x) = \top$ for some critic x. Formally:

$$[\![\text{every film}]\!]^k = \textsc{every}(\textsc{film})(k), \qquad [\![\text{a critic}]\!]^k = \textsc{some}(\textsc{critic})(k).$$

What about expressions whose meanings are, unlike quantifiers, not inherently scopal? In our continuized semantics, any such M simply denotes the property that holds of k iff $k(M) = \top$. For example,

$$[\![\text{John}]\!]^k = k(\textsc{john}), \qquad [\![\text{panned}]\!]^k = k(\textsc{panned}).$$

Going forward, it will be useful to make an essentially cosmetic change and write k not as a superscript to the denotation brackets but as an explicit argument of an expression's denotation, e.g.:

$$[\![\text{every film}]\!] = \lambda k^{\text{et}}.\textsc{every}(\textsc{film})(k),$$
$$[\![\text{John }]\!] = \lambda k.k^{\text{et}}(\textsc{john}).$$

So far, nothing new has really happened. The denotations of the quantifiers given above are the same as the traditional ones, and the denotations of all other expressions are just the Montague lift of their traditional meanings (i.e., the result of applying the type-shifter $\lambda M\lambda k.kM$). The real innovation of continuations comes from an upgraded notion of semantic combination. Suppose that a node of a tree has two daughters, X and Y, and that '$\star$' is some binary mode of combination (e.g. forward application '>', backward application '<', or predicate modification '$\sqcap$'). Then we can combine the denotations of X and Y as follows (assuming that this expression is well-typed):

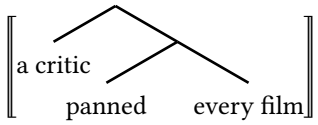
$$\left[\!\!\left[\begin{array}{ccc} & \diagup\diagdown & \\ X & & Y \end{array}\right]\!\!\right] = \lambda k.[\![X]\!](\lambda x.[\![Y]\!](\lambda y.k(x \star y))).$$

I call this mode of combination $\mathbf{K}(\star)$.

This machinery is already sufficient to interpret sentences with quantifiers. For example, using $\mathbf{K}(>)$ we can assign a meaning to the verb phrase *panned every film*:

$$\begin{aligned}
\left[\!\!\left[\begin{array}{ccc} & \diagup\diagdown & \\ \text{panned} & & \text{every film} \end{array}\right]\!\!\right] &= \lambda k^{(\text{et})\text{t}}.[\![\text{panned}]\!]\big(\lambda x^{\text{et}}.[\![\text{every film}]\!](\lambda y^{\text{e}}.k(x\,y))\big) \\
&= \lambda k^{(\text{et})\text{t}}.\Big(\lambda c^{(\text{eet})\text{t}}.c(\textsc{panned})\Big)\big(\lambda x^{\text{eet}}.[\![\text{every film}]\!](\lambda y^{\text{e}}.k(x\,y))\big) \\
&= \lambda k^{(\text{et})\text{t}}.[\![\text{every film}]\!](\lambda y^{\text{e}}.k(\textsc{panned}\ y)) \\
&= \lambda k^{(\text{et})\text{t}}.\big(\lambda c^{\text{et}}.\textsc{every}(\textsc{film})(c)\big)(\lambda y^{\text{e}}.k(\textsc{panned}\ y)) \\
&= \lambda k^{(\text{et})\text{t}}.\textsc{every}(\textsc{film})(\lambda y^{\text{e}}.k(\textsc{panned}\ y)).
\end{aligned}$$

We can then combine this expression with *a critic* using $\mathbf{K}(<)$:



$= \lambda k^{\mathrm{tt}}.[\![\text{a critic}]\!]\big(\lambda x^{\mathrm{e}}.[\![\text{panned every film}]\!]\big(\lambda y^{\mathrm{et}}.k(yx)\big)\big)$

$= \lambda k^{\mathrm{tt}}.\big(\lambda c^{\mathrm{et}}.\textsc{some}(\textsc{critic})(c)\big)\big(\lambda x^{\mathrm{e}}.[\![\text{panned every film}]\!]\big(\lambda y^{\mathrm{et}}.k(yx)\big)\big)$

$= \lambda k^{\mathrm{tt}}.\textsc{some}(\textsc{critic})\big(\lambda x^{\mathrm{e}}.[\![\text{panned every film}]\!]\big(\lambda y^{\mathrm{et}}.k(yx)\big)\big)$

$= \lambda k^{\mathrm{tt}}.\textsc{some}(\textsc{critic})\Big(\lambda x^{\mathrm{e}}.\big(\lambda c^{(\mathrm{et})\mathrm{t}}.\textsc{every}(\textsc{film})(\lambda y^{\mathrm{e}}.c(\textsc{panned}\ y))\big)\big(\lambda y^{\mathrm{et}}.k(yx)\big)\Big)$

$= \lambda k^{\mathrm{tt}}.\textsc{some}(\textsc{critic})(\lambda x^{\mathrm{e}}.\textsc{every}(\textsc{film})\big(\lambda y^{\mathrm{et}}.k(\textsc{panned}\ yx)\big)).$

The result is a property of contexts. To reduce it to a truth value, we can evaluate it at the empty context by passing in the identity function $\lambda x.x$ as the continuation argument k, yielding: $\textsc{some}(\textsc{critic})(\lambda x^{\mathrm{e}}.\textsc{every}(\textsc{film})(\lambda y^{\mathrm{e}}.\textsc{panned}\ yx))$. This expression represents the linear scope reading of the sentence.

These calculations become significantly more readable with a notational convention for λ-terms popularized by Barker and Shan (2014), who write expressions of the form $\lambda k.\ldots kx\ldots$ as a *tower*: $\dfrac{\ldots[\,]\ldots}{x}$. In this new notation, the denotations for *a critic* and for *panned every film*, for example, are written like this:

$$\frac{\textsc{some}(\textsc{critic})(\lambda x^{\mathrm{e}}.[\,])}{x} \quad \text{and} \quad \frac{\textsc{every}(\textsc{film})(\lambda y^{\mathrm{e}}.[\,])}{\textsc{panned}\,y}$$

Given a mode of combination '$\star$', we can express the continuized mode of combination $\mathbf{K}(\star)$ very simply in terms of towers:

$$\left(\frac{F[\,]}{x}\right)\left(\frac{G[\,]}{y}\right) = \frac{F[G[\,]]}{x \star y}$$

This schema makes it easy to combine *a critic* with *panned every film* using $\mathbf{K}(<)$:

$$\left(\frac{\textsc{some}(\textsc{critic})(\lambda x^{\mathrm{e}}.[\,])}{x}\right)\left(\frac{\textsc{every}(\textsc{film})(\lambda y^{\mathrm{e}}.[\,])}{\textsc{panned}\,y}\right)$$

$$= \frac{\textsc{some}(\textsc{critic})(\lambda x^{\mathrm{e}}.\textsc{every}(\textsc{film})(\lambda y^{\mathrm{e}}.[\,]))}{\textsc{panned}\ yx}$$

The resulting denotation is the same as the one computed earlier, but with the tower notation, it's possible to skip the tedious intermediate β-reductions.

With those preliminaries out of the way, we can now derive the inverse scope reading of *a critic panned every film*. The trick is to make judicious use of the Montague lift type-shifter, $\mathbf{U} = \lambda M\lambda k.kM$. The structure we will interpret is shown in Figure 7. The denotation of the root's left daughter is a three-story tower:

$$\mathbf{U}([\![\text{a critic}]\!]) = \lambda k^{((\mathrm{et})\mathrm{t})\mathrm{t}}.k\big(\lambda c^{\mathrm{et}}.\textsc{some}(\textsc{critic})(\lambda x^{\mathrm{e}}.cx)\big) = \cfrac{\cfrac{[\,]}{\textsc{some}(\textsc{critic})(\lambda x.[\,])}}{x}$$

The denotation of the node $\mathbf{U}(\mathbf{U})$ is just the two-story tower $\dfrac{[\,]}{\mathbf{U}} = \lambda k.k(\mathbf{U})$. We can then combine this with *panned every film* using $\mathbf{K}(>)$ to get the denotation of the root's right

daughter. The effect of combining with $\mathbf{U}(\mathbf{U})$ is to add an extra level to a tower directly below the top level:

$$\left(\frac{[\,]}{\mathbf{U}}\right)\left(\frac{\textsc{every}(\textsc{film})(\lambda y^{\mathrm{e}}.[\,])}{\textsc{panned}\,y}\right) = \frac{\textsc{every}(\textsc{film})(\lambda y^{\mathrm{e}}.[\,])}{\mathbf{U}(\textsc{panned}\,y)}$$

$$= \frac{\dfrac{\textsc{every}(\textsc{film})(\lambda y^{\mathrm{e}}.[\,])}{[\,]}}{\textsc{panned}\,y}$$

Finally we can combine the root's left daughter and right daughter using $\mathbf{K}(\mathbf{K}(<))$:

$$\left(\frac{\dfrac{[\,]}{\textsc{some}(\textsc{critic})(\lambda x^{\mathrm{e}}.[\,])}}{x}\right)\left(\frac{\textsc{every}(\textsc{film})(\lambda y^{\mathrm{e}}.[\,])}{\textsc{panned}\,y}\right) = \frac{\textsc{every}(\textsc{film})(\lambda y^{\mathrm{e}}.[\,])}{\textsc{some}(\textsc{critic})(\lambda x^{\mathrm{e}}.[\,])}$$

How do we lower this tower and obtain a truth value? Earlier, we lowered a two-story tower by passing it the identity function as its continuation. Now, to lower a three-story tower, we pass it a *lifted* identity function $\mathbf{U}(\lambda x.x) = \lambda k.k(\lambda x.x)$ as its continuation. When the dust settles, we're left with the following meaning, which represents exactly the inverse scope interpretation we were after:

$$\textsc{every}(\textsc{film})(\lambda y^{\mathrm{e}}.\textsc{some}(\textsc{critic})(\lambda x^{\mathrm{e}}.\textsc{panned}\;y\,x)).$$

Continuations are a powerful alternative to standard techniques like QR. We've seen, for example, how a continuized semantics can interpret constituents that seem to contain a type clash, like *panned every film*, and can combine a scope-taker with its scope without any need for syntactic manipulation or a Cooper store. That said, any analysis using continuations still has to answer the same basic question that QR and Cooper Storage analyses do, viz., what takes scope where? The answer for any particular sentence is determined by the way the lift type-shifter $\mathbf{U}$ is used. The examples above showed, for instance, how the presence or absence of lifts determine whether *every film* scopes over a quantifier to its left or not. In general, unconstrained application of $\mathbf{U}$ is enough to generate every possible arrangement of the scope-takers in a sentence. This flexibility is useful, but it also means that the space of possible derivations for a sentence in a continuized semantics can—just like with QR and Cooper Storage—be large and perhaps computationally difficult to explore.

3.4. Summary

This section presented three different ways to give an expression scope over part of its semantic context. Which method one chooses has theoretical implications that are important for semanticists to reckon with. Continuations, for example, adhere to the appealing methodological principle of direct compositionality (Jacobson 2012). But they

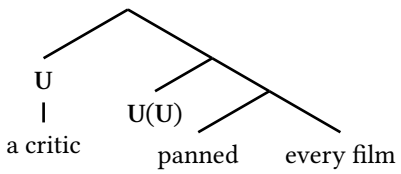


Figure 7: Inverse scope reading of *a critic panned every film*.

also require moving to an enriched notion of meaning according to which expressions denote properties of contexts. The presentation of Cooper Storage above is directly compositional as well, but likewise requires a different view of meanings according to which they are pairs of a regular denotation and a quantifier store. QR, which involves syntactic manipulation of a surface structure and introduces object-language variables, is not directly compositional, but it does maintain a simple notion of meaning and a straightforward semantic combinatorics.

Nonetheless, from the narrower perspective of computational tractability, the three techniques are more alike than different. In particular, they are similar in that they place essentially no constraints on where an expression takes scope beyond the simple requirement that the result be well typed. The rest of this paper is devoted to QR because it is the most popular scope-taking mechanism and because it lends itself to a clean formalization as a decision problem. But the basic computational problem of finding a well-typed arrangement of the scope-takers in a sentence is inherent to Cooper Storage and continuations too. The upcoming intractability result about QR should therefore have a precise analogue in these two other frameworks.

4. Formal Preliminaries and Notation

This section begins the the mathematical portion of the paper. It defines Logical Forms and Quantifier Raising, formalizes the problem of finding a well-typed LF as a decision problem called LF REPAIR, and proves a couple of theorems that are needed later on.

4.1. Types, Logical Forms, Contexts

Given a finite set $\mathbb{A}$ of atomic types, the set of *types* over it is

$$\mathbb{T} ::= \mathbb{A} \mid \mathbb{T} \to \mathbb{T}.$$

In the rest of this paper, lowercase Greek letters ($\alpha, \beta, \gamma, \ldots$) will range over types. As usual, '$\to$' associates to the right. I often drop the '$\to$', writing $\alpha\beta\gamma$, for example, as a shorthand for the type $\alpha \to (\beta \to \gamma)$. I also use $\alpha \overset{m}{\to} \beta$ as a shorthand for

$$\underbrace{\alpha \to \cdots \to \alpha}_{m \text{ times}} \to \beta.$$

A useful way to stratify types is by assigning them an *order*. If we think of types as describing functions, then the order quantifies how complex that function's arguments are. Formally, the order of an atomic type p is $\mathrm{ord}(p) = 1$, and the order of a complex type $\alpha \to \beta$ is

$$\mathrm{ord}(\alpha \to \beta) = \max(1 + \mathrm{ord}(\alpha), \mathrm{ord}(\beta)).$$

Types of order n therefore describe functions whose arguments have types of order less than n. Some examples: if $\mathrm{e}, \mathrm{t} \in \mathbb{A}$, then $\mathrm{ord}(\mathrm{e}) = 1$, $\mathrm{ord}(\mathrm{eet}) = 2$, and $\mathrm{ord}((\mathrm{et})\mathrm{et}) = 3$.

Let $\{\mathbb{V}_\alpha\}_{\alpha \in \mathbb{T}}$ be a disjoint family of countable sets indexed by $\mathbb{T}$, and let

$$\mathbb{V} = \bigcup_{\alpha \in \mathbb{T}} \mathbb{V}_\alpha.$$

The members of each set $\mathbb{V}_\alpha = \{x, y, \ldots\}$ are *variables of type* α. Nodding at linguistic practice, I sometimes call variables *traces*. When the type of a variable x is relevant, I will write it as x^α to indicate that $x \in \mathbb{V}_\alpha$.

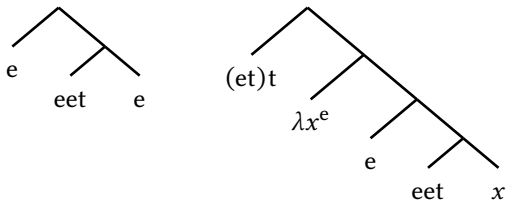
The set $\mathbb{L}$ of *logical forms* is

$$\mathbb{L} ::= \mathbb{T} \mid \mathbb{V} \mid \mathbb{L} \bullet \mathbb{L} \mid \lambda \mathbb{V}.\mathbb{L}.$$

In words, an LF is either a type, a variable, the concatenation via the binary operator '•' of two LFs, or the combination via the binary operator '.' of a λ-abstractor and an LF. An *abstraction-free* LF is one built from only the first three cases, i.e., an LF that contains no λ-abstractions. It's usual in semantics to place typed lexical items rather than types themselves at the leaves of logical forms. But since this paper is only concerned with type coherence, the definition above is simpler.

The constructor '•' associates to the left and '.' to the right. Both are commutative, meaning $\alpha \bullet \beta$ and $\beta \bullet \alpha$ are considered equivalent, as are $\lambda x.\alpha$ and $\alpha.\lambda x$. The '.' binds more tightly than '•'.

If $\mathrm{e}, \mathrm{t} \in \mathbb{A}$, then $\mathrm{e} \bullet (\mathrm{eet} \bullet \mathrm{e})$ and $(\mathrm{et})\mathrm{t} \bullet \lambda x^{\mathrm{e}}.(\mathrm{e} \bullet (\mathrm{eet} \bullet x^{\mathrm{e}}))$ are each examples of logical forms. I sometimes present LFs as unordered binary trees:



Later on, I will need to talk about LFs with a "hole" that can be plugged by another LF. I call these *contexts*. Formally, the set of contexts is

$$\mathbb{C} ::= [\,] \mid \mathbb{L} \bullet \mathbb{C} \mid \lambda \mathbb{V}.\mathbb{C}.$$

If Γ is a context and Δ is an LF (or context), then $\Gamma[\Delta]$ is the LF (or context) that results from substituting Δ for $[\,]$ in Γ. In Section 6, it will be useful to have some notation for repeatedly nesting a context inside itself: if Γ is a context, let $\Gamma^0 = [\,]$ and $\Gamma^n = \Gamma\left[\Gamma^{n-1}\right]$. In the rest of the paper, capital Greek letters (Γ, Δ, etc.) will denote either LFs or contexts.

The *typing function* $\tau : \mathbb{L} \to \mathbb{T}$ is the following partial map:

$$\tau(\Gamma) = \begin{cases} \alpha & \text{if } \alpha \text{ is a type and } \Gamma = \alpha, \\ \beta & \text{if } \Gamma = \Delta \bullet \Theta,\ \tau(\Delta) = \alpha\beta, \text{ and } \tau(\Theta) = \alpha, \\ \alpha\beta & \text{if } \Gamma = \lambda x^{\alpha}.\Delta[x^{\alpha}] \text{ and } \tau(\Delta[\alpha]) = \beta, \\ \text{undefined} & \text{otherwise.} \end{cases}$$

The second and third cases correspond respectively to the semantic rules of *Function Application* and *Predicate Abstraction* in Heim and Kratzer (1998). When $\tau(\Gamma) = \alpha$ holds, I will usually just say Γ has type α. I occasionally use the notation $\Gamma : \alpha$ to mean that either $\tau(\Gamma) = \alpha$ or $\Gamma = x$ for some $x \in \mathbb{V}_{\alpha}$. (Note that $\tau(x^{\alpha})$ is formally undefined.)

An LF Γ is *well typed* iff $\tau(\Gamma)$ is defined. The two LFs mentioned above,

$$\mathrm{e} \bullet (\mathrm{eet} \bullet \mathrm{e}) \quad \text{and} \quad (\mathrm{et})\mathrm{t} \bullet \lambda x^{\mathrm{e}}.(\mathrm{e} \bullet (\mathrm{eet} \bullet x^{\mathrm{e}})),$$

each have type t and are therefore well typed.

A trace x in an LF is *bound* if it occurs within a subtree of the form $\lambda x.\Sigma[x]$. Otherwise it's *unbound*. Notice that an LF containing an unbound trace cannot be well typed. The definition of the typing function therefore enforces a form of the empirically motivated proposal that remnant movement must reconstruct (Huang 1993; Bhatt and Dayal 2007): movement steps that create an unbound trace are always "undone" before interpretation begins.

4.2. Formalizing QR

In order to study QR in its most general form, my definition won't constrain it by—just to name a few possibilities—imposing scope islands, scope economy (Fox 1995), or limits on the types of traces (Landman 2006; Poole 2024). The methodological reason for this choice is simple: to understand the effect (if any) that specific constraints have on complexity, it's necessary to first understand the complexity of the general case.

That said, *one* formal constraint is necessary to keep QR from running amok: the operation should be disallowed from targeting abstractions, that is, LFs of the form $\lambda x.\Gamma$. To illustrate what can go wrong without this constraint, look at Figure 8.

The leftmost tree shows an innocent-looking analysis of the sentence *Mary gave John the book*. It might not seem like QR can do anything interesting to this tree, because no subtree has a sufficiently complex type to take scope. But the ability to raise abstractions lets us *create* a scope-taker.

The middle tree is the result of applying QR to *gave*, moving it to a position right below *Mary* and leaving a trace of type eet. The rightmost tree is the result of quantifier raising the *abstraction* created by the previous step to the root of the tree and leaving a trace of type e. Not only is the resulting LF well typed, but (assuming natural denotations for the lexical items) it means *the book gave John Mary*.

Completely unrestricted QR therefore allows us to create scope-takers on the fly, even in sentences where nothing ought to be taking scope, and the semantic effect is disastrous. It does not behave in the way linguists expect QR to behave and it lacks good theoretical properties. The definition below therefore explicitly prevents QR from targeting abstractions.[2]

Definition 4.2.1. *Quantifier Raising* is the smallest binary relation $\to_{\text{QR}} \subseteq \mathbb{L} \times \mathbb{L}$ such that for all contexts Γ and Δ, all LFs Θ that are *not* abstractions, and all fresh variables x,

$$\Gamma[\Delta[\Theta]] \to_{\text{QR}} \Gamma[\Theta \bullet \lambda x.\Delta[x]].$$

The reflexive, transitive closure of $\to_{\text{QR}}$ is $\to^*_{\text{QR}}$.

Now we're in a position to define the central decision problem of this paper:

> **LF REPAIR**
> Given a set of atomic types $\mathbb{A}$, an abstraction-free LF Γ, and a type α, is there a Δ of type α such that $\Gamma \to^*_{\text{QR}} \Delta$?

The name comes from the idea that ill-typed LFs are "broken," and that repairing them involves raising the scope-takers and finding appropriate trace types in such a way that the result is well-typed.

Here is an example. Let $\mathbb{A} = \{\mathrm{e}, \mathrm{t}\}$ and let Γ be the ill-typed LF

$$[(\mathrm{et})\mathrm{e} \bullet [(((\mathrm{et})\mathrm{et})\mathrm{et})\mathrm{et} \bullet \mathrm{et}]] \bullet [\mathrm{eet} \bullet (\mathrm{et})\mathrm{t}].$$

Is there a Δ of type t such that $\Gamma \to^*_{\text{QR}} \Delta$? The answer, not immediately obvious, is yes:

2. Another, maybe more elegant way to ban QR of abstractions is to change the structure of LFs so that no constituent corresponds to an abstraction in the first place (Heim and Kratzer 1998, 187–88). Variable binding would instead be handled by an index on moved items, so that QR would relate an LF $\Gamma[\Delta[\Theta]]$ to something like $\Gamma[\Theta_x \bullet \Delta[x]]$ (see, for example, Kobele 2010). But this definition would also rule out the possibility of *parasitic scope* (Barker 2007), so I've chosen the more general option above.

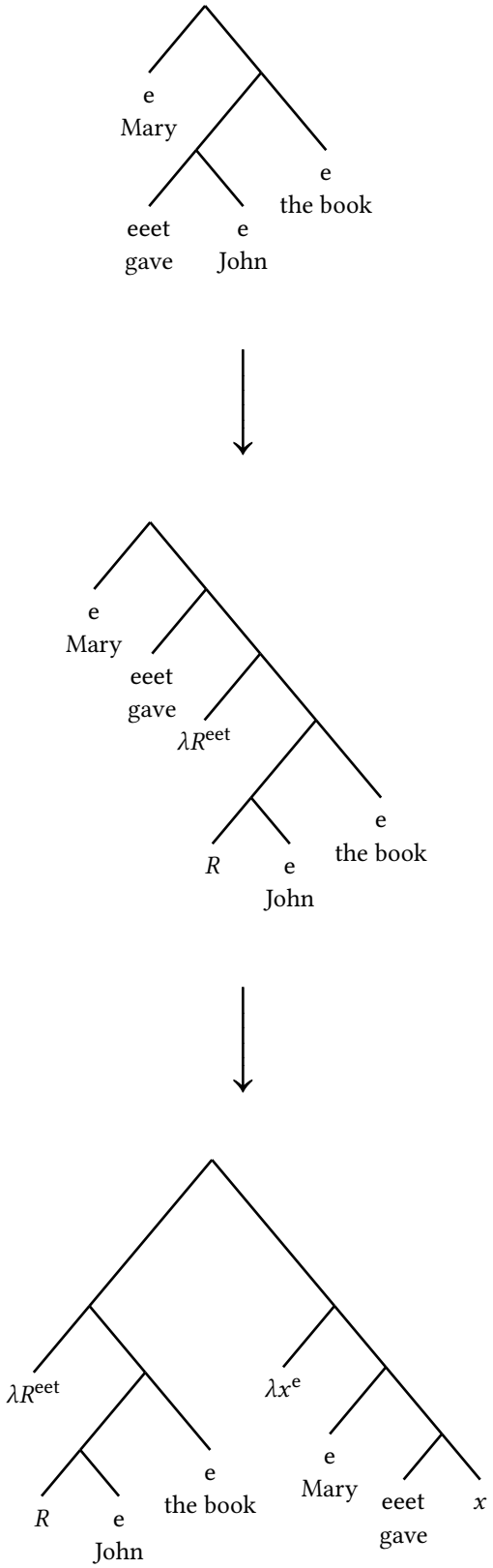


Figure 8: Raising abstractions is dangerous.

$$\begin{aligned}\Gamma &\to_{\mathrm{QR}} (\mathrm{et})\mathrm{t} \bullet \lambda x^{\mathrm{e}}[(\mathrm{et})\mathrm{e} \bullet [(((\mathrm{et})\mathrm{et})\mathrm{et})\mathrm{et} \bullet \mathrm{et}]] \bullet [\mathrm{eet} \bullet x^{\mathrm{e}}] \\ &\to_{\mathrm{QR}} (\mathrm{et})\mathrm{t} \bullet \Big[(((\mathrm{et})\mathrm{et})\mathrm{et})\mathrm{et} \bullet \lambda Q^{((\mathrm{et})\mathrm{et})} \lambda x^{\mathrm{e}} \Big[(\mathrm{et})\mathrm{e} \bullet \Big[Q^{((\mathrm{et})\mathrm{et})} \bullet \mathrm{et}\Big]\Big] \bullet [\mathrm{eet} \bullet x^{\mathrm{e}}]\Big],\end{aligned}$$

and this last LF has type t.

4.3. Normalization

The purpose of QR is to allow a function to take some portion of its semantic context as an argument. But it's also possible that the element raised by QR ends up an argument of its context, rather than the other way around: for example, $\mathrm{e} \bullet \mathrm{et} \to_{\mathrm{QR}} \mathrm{e} \bullet \lambda x^{\mathrm{e}}(x^{\mathrm{e}} \bullet \mathrm{et})$. Such steps are licit but, as far as typability is concerned, pointless—they will never help turn an ill-typed LF into a well-typed one. In general, if an LF has the form $\Delta[\Pi \bullet \lambda x.\Sigma[x]]$, where $\Pi : \alpha$ and $\lambda x.\Sigma[x]$ has type $\alpha\beta$, then I call the abstraction $\lambda x.\Sigma[x]$ *eliminable*. A well-typed LF is *normalized* iff it contains no eliminable abstractions.

The next lemma justifies the label "eliminable."

Lemma 4.1. Let Γ be an abstraction-free LF. If $\Gamma \to^*_{\mathrm{QR}} \Delta$, where Δ has type α, then there exists a normalized LF Δ' of type α such that $\Gamma \to^*_{\mathrm{QR}} \Delta'$.

Proof: If Δ isn't already normalized, then it contains a eliminable abstraction and therefore has the form $\Theta\Big[\Pi \bullet \lambda x^{\beta}\, \Sigma\Big[x^{\beta}\Big]\Big]$, where $\Pi : \beta$ and $\lambda x^{\beta}\, \Sigma\Big[x^{\beta}\Big]$ has type $\beta\gamma$. By definition, QR cannot target an abstraction. And since Δ is well-typed, no step in the derivation $\Gamma \to^*_{\mathrm{QR}} \Delta$ raises x^{β} outside the scope of its binder λx^{β}. (If x *were* raised outside its binder, then it would become a free variable. Since no QR step can bind a free variable, x^{β} would remain free in Δ, which would, contrary to assumption, therefore not be well-typed.)

The derivation $\Gamma \to^*_{\mathrm{QR}} \Delta$ therefore has the following form:

$$\begin{aligned}\Gamma &\to^*_{\mathrm{QR}} \Theta_1[\Sigma_1[\Pi_1]] \\ &\to_{\mathrm{QR}} \Theta_1\Big[\Pi_1 \bullet \lambda x^{\beta}\, \Sigma_1\Big[x^{\beta}\Big]\Big] \\ &\to_{\mathrm{QR}} \Theta_2\Big[\Pi_2 \bullet \lambda x^{\beta}\, \Sigma_2\Big[x^{\beta}\Big]\Big] \\ &\to_{\mathrm{QR}} \cdots \\ &\to_{\mathrm{QR}} \Theta_n\Big[\Pi_n \bullet \lambda x^{\beta}\, \Sigma_n\Big[x^{\beta}\Big]\Big],\end{aligned}$$

where $\Theta_n = \Theta$, $\Sigma_n = \Sigma$, and $\Pi_n = \Pi$. The following is therefore a valid derivation of $\Theta[\Sigma[\Pi]]$:

$$\begin{aligned}\Gamma &\to^*_{\mathrm{QR}} \Theta_1[\Sigma_1[\Pi_1]] \\ &\to_{\mathrm{QR}} \Theta_2[\Sigma_2[\Pi_2]] \\ &\to_{\mathrm{QR}} \cdots \\ &\to_{\mathrm{QR}} \Theta_n[\Sigma_n[\Pi_n]]\end{aligned}$$

Like Δ, the LF $\Theta[\Sigma[\Pi]]$ also has type α, but it contains one fewer eliminable abstraction. By repeating this process, we can remove eliminable abstractions one by one until we obtain a normalized LF Δ' of type α such that $\Gamma \to^*_{\mathrm{QR}} \Delta'$. ∎

The following lemma, needed in Section 6, makes it easier to check whether an abstraction-free LF Γ can be repaired. It turns out that the order of the traces introduced by each QR step never has to exceed a constant fixed by Γ.

Lemma 4.2. Suppose Γ is an abstraction-free LF, Δ is a normalized LF, and $\Gamma \rightarrow^*_{\text{QR}} \Delta$. Let m be the maximum order among Γ's leaves. If x^α is a trace occurring in Δ, then $\text{ord}(\alpha) < m - 1$.

Proof: Let n be the maximum order among all of Δ's traces, and suppose $n \geq m - 1$. Consider a trace x^α of order n and write

$$\Delta = \Theta[\Pi \bullet \lambda x^\alpha \, \Sigma[x]]. \tag{1}$$

(Technically, there could be more abstractions intervening, so that I should write

$$\Theta\Big[\Pi \bullet \lambda y_1^{\beta_1} \cdots \lambda y_k^{\beta_k} \lambda x^\alpha \, \Sigma[x^\alpha]\Big], \tag{2}$$

but this doesn't affect the rest of the proof.)

Because Δ is normalized, Π's type has the form $(\alpha \rightarrow \tau(\Sigma[\alpha])) \rightarrow \beta$ for some β, and so

$$\text{ord}(\tau(\Pi)) > \text{ord}(\alpha) + 1 = n + 1. \tag{3}$$

Every node within the subtree Π is either (i) a leaf of order at most $m \leq n + 1$, (ii) function application of two elements of order at most $n + 1$, or (iii) an abstraction over a variable of order at most n. It follows (by an induction from the leaves to the root) that every node of Π has order at most $n + 1$, so $\text{ord}(\tau(\Pi)) \leq n + 1$. This contradicts Equation 3. Therefore $n < m - 1$. ∎

5. LF REPAIR is in NP

The goal of this section is to prove that LF REPAIR is in NP. This result would be a simple corollary of the proof in Barker (2020) that LF REPAIR is decidable, but it turns out the proof is not valid.[3] The rest of this section provides a corrected proof, and uses it to show that LF REPAIR is in NP. Because many of the details from Barker's paper don't need to change, this section moves fast. See his paper for further information.

The plan, following Barker's lead, is to introduce a type-logical grammar called *Quantifier Raising with Types* (QRT) that captures the way QR manipulates Logical Forms. Theorems of the logic have the form $\Gamma \vdash A$, where Γ is an LF and A is a (possibly non-atomic) type. The main reason why QRT captures the logic of Quantifier Raising is that for every abstraction-free LF Γ, the following are equivalent:[4]

- $\Gamma \vdash A$ is a theorem of QRT,
- $\Gamma \rightarrow^*_{\text{QR}} \Delta$ for some Δ of type A.

The natural interpretation of the theorem $\Gamma \vdash A$ is accordingly that Γ can be assigned type A after some number of applications of QR. An immediate consequence is that LF REPAIR has the same time complexity as provability in QRT. Since Theorem 5.3 establishes that QRT-provability is in NP, it follows that LF REPAIR belongs to NP too.

The axiom and inference rules of QRT are shown in Figure 9. Γ, Δ and Θ represent LFs or contexts, and A and B represent types. The variable in the rules $\rightarrow$R and $\lambda^\downarrow$ is required to be fresh. The rules $\lambda^\uparrow$ and $\lambda^\downarrow$ are subject to the side condition that the LF Θ not be a λ-abstraction (or equivalently, that Θ be either a type, a variable, or of the form $\Sigma \bullet \Omega$). This restriction, which is the only difference between the version of QRT in this paper and

3. Barker (2020) points out the problem himself in footnote 1, added after the early-access version of the paper was published, and suggests a way to overcome it. Appendix A of this paper discusses his proposal.
4. For a proof of this equivalence which carries over with minimal changes to the version of QRT in this paper, see the appendix to Barker (2020).

$$\frac{}{A \vdash A}\,\text{Ax} \qquad \frac{\Gamma \vdash A \quad \Delta[B] \vdash C}{\Delta[\Gamma \bullet AB] \vdash C}\,{\to}\text{L} \qquad \frac{\Gamma[A] \vdash B}{\lambda x.\Gamma[x] \vdash AB}\,{\to}\text{R}$$

$$\frac{\Gamma[\Theta \bullet \lambda x.\Delta[x]] \vdash A}{\Gamma[\Delta[\Theta]] \vdash A}\,\lambda^{\uparrow} \qquad \frac{\Gamma[\Delta[\Theta]] \vdash A}{\Gamma[\Theta \bullet \lambda x.\Delta[x]] \vdash A}\,\lambda^{\downarrow} \qquad \frac{\Gamma \vdash A \quad \Delta[A] \vdash B}{\Delta[\Gamma] \vdash B}\,\textsc{Cut}$$

Figure 9: Axiom and inference rules of QRT.

the one in Barker (2020), will guarantee that the critical Theorem 5.2 holds. Unrestricted raising—the source of problems in Barker (2020)—creates trouble when combined with the commutativity of '•'. For instance, it lets us freely swap an inner and outer context above a '•'-node, which results in massive overgeneration:

$$\dfrac{\dfrac{\dfrac{\dfrac{\Gamma[\Delta[A \bullet B]] \vdash C}{\Gamma[A \bullet \lambda x.\Delta[x \bullet B] \vdash C]}\,\lambda^{\downarrow}}{(\lambda x.\Delta[x \bullet B]) \bullet \lambda y.\Gamma[A \bullet y] \vdash C}\,\lambda^{\downarrow}}{\Delta[(\lambda y.\Gamma[A \bullet y]) \bullet B] \vdash C}\,\lambda^{\uparrow}}{\Delta[\Gamma[A \bullet B]] \vdash C}\,\lambda^{\uparrow}$$

This issue is not really different from the one in Figure 8, and my solution is similar: forbidding $\lambda^{\uparrow}$ and $\lambda^{\downarrow}$ from targeting abstractions.

Even though every variable in an LF has a fixed type, QRT ignores this fact: any variable of type α in a QRT proof could be uniformly replaced by a variable of type β without affecting correctness. Put another way, QRT only sees LFs up to α-equivalence. The real meaning of $\Gamma \vdash A$ is that there is *some* way to assign types to the variables in Γ such that Γ can be given type A by repeatedly applying QR. More formally, there exists a Γ' differing from Γ only in its variables such that $\Gamma' \to_{\text{QR}}^{*} \Delta$ for a Δ of type A.

Here is a small QRT proof, showing that a sentence with a quantificational subject, a transitive verb, and a quantificational object can be given type t:

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\text{e} \vdash \text{e}}\,\text{Ax} \quad \dfrac{\dfrac{}{\text{et} \vdash \text{et}}\,\text{Ax} \quad \dfrac{}{\text{t} \vdash \text{t}}\,\text{Ax}}{(\text{et})\text{t} \bullet \text{et} \vdash t}\,{\to}\text{L}}{\lambda x.((\text{et})\text{t} \bullet (\text{eet} \bullet x)) \vdash \text{et}}\,{\to}\text{R} \quad \dfrac{}{\text{t} \vdash \text{t}}\,\text{Ax}}{(\text{et})\text{t} \bullet \lambda x.((\text{et})\text{t} \bullet (\text{eet} \bullet x)) \vdash \text{t}}\,{\to}\text{L}}{(\text{et})\text{t} \bullet (\text{eet} \bullet (\text{et})\text{t}) \vdash \text{t}}\,\lambda^{\uparrow}$$

There is a natural decision problem associated with QRT. As mentioned above, it has exactly the same time complexity as LF REPAIR.

QRT PROVABILITY
Given some abstraction-free sequent, is it a theorem of QRT?

The rest of this section will prove that this problem is in NP.

The first step in the proof is to prove that cut-elimination is possible in proofs of QRT sequents that are *abstraction-free*, that is, sequents $\Gamma \vdash A$ where Γ contains no λ-abstractions.

Theorem 5.1. Every abstraction-free theorem of QRT has a CUT-free proof

Proof: By the normal method of permuting cuts upward. There is a fair number of cases to consider, and most of them are routine, so the proof below only shows some of them.

Suppose the left premise of Cut was introduced by →L.

$$\dfrac{\dfrac{\Gamma[B] \vdash C \qquad \Delta \vdash A}{\Gamma[AB \bullet \Delta] \vdash C}\to\mathrm{L} \qquad \Theta[C] \vdash D}{\Theta[\Gamma[AB \bullet \Delta]] \vdash D}\,\textsc{Cut}$$

Then we can permute the application of Cut upwards as follows:

$$\dfrac{\Gamma[B] \vdash C \qquad \dfrac{\Theta[C] \vdash D}{\Theta[\Gamma[B]] \vdash D} \qquad \Delta \vdash A}{\Theta[\Gamma[AB \bullet \Delta]] \vdash D}\to\mathrm{L}$$

The following proof transformations handle cases where the left premise is introduced by $\lambda^{\uparrow}$ and $\lambda^{\downarrow}$:

$$\dfrac{\dfrac{\Gamma[\Delta[\Theta]] \vdash A}{\Gamma[\Theta \bullet \lambda x.\Delta[x]] \vdash A}\lambda^{\downarrow} \qquad \Lambda[A] \vdash B}{\Lambda[\Gamma[\Theta \bullet \lambda x.\Delta[x]]] \vdash B}\,\textsc{Cut} \quad\rightsquigarrow\quad \dfrac{\Gamma[\Delta[\Theta]] \vdash A \qquad \Lambda[A] \vdash B}{\Lambda[\Gamma[\Delta[\Theta]]] \vdash B}\,\textsc{Cut}}{\Lambda[\Gamma[\Theta \bullet \lambda x.\Delta[x]]] \vdash B}\lambda^{\downarrow}$$

$$\dfrac{\dfrac{\Gamma[\Theta \bullet \lambda x.\Delta[x]] \vdash A}{\Gamma[\Delta[\Theta]] \vdash A}\lambda^{\uparrow} \qquad \Lambda[A] \vdash B}{\Lambda[\Gamma[\Delta[\Theta]]] \vdash B}\,\textsc{Cut} \quad\rightsquigarrow\quad \dfrac{\Gamma[\Theta \bullet \lambda x.\Delta[x]] \vdash A \qquad \Lambda[A] \vdash B}{\Lambda[\Gamma[\Theta \bullet \lambda x.\Delta[x]]] \vdash B}\,\textsc{Cut}}{\Lambda[\Gamma[\Delta[\Theta]]] \vdash B}\lambda^{\uparrow}$$

When the left premise is introduced by →R, it becomes necessary to consider which rule introduced the right premise. In most cases, the transformations are straightforward. The one subtle situation is when the cut formula itself is introduced in the right premise by →L:

$$\dfrac{\dfrac{\Gamma[A] \vdash B}{\lambda x.\Gamma[x] \vdash AB}\to\mathrm{R} \qquad \dfrac{\Delta \vdash A \qquad \Theta[B] \vdash C}{\Theta[AB \bullet \Delta] \vdash C}\to\mathrm{L}}{\Theta[\lambda x.\Gamma[x] \bullet \Delta] \vdash C}\,\textsc{Cut}$$

Notice that Δ cannot in this case be an abstraction. To see why, first define a sequent to be *stuck* if its left-hand side has the form $\Sigma[\Phi \bullet \Psi]$ where Φ and Ψ are both abstractions. A straightforward induction shows that if a sequent in a QRT proof is stuck, then every subsequent sequent—and in particular the conclusion—is stuck too. Now if Δ *were* an abstraction, then $\Theta[(\lambda x.\Gamma[x]) \bullet \Delta] \vdash C$ would be stuck and the proof's conclusion would contain an abstraction, contrary to assumption. Δ is therefore not an abstraction, so $\lambda^{\downarrow}$ may target it and we can replace the Cut with two smaller ones:

$$\dfrac{\dfrac{\Delta \vdash A \qquad \Gamma[A] \vdash B}{\Gamma[\Delta] \vdash B}\,\textsc{Cut} \qquad \Theta[B] \vdash C}{\Theta[\Gamma[\Delta]] \vdash C}\,\textsc{Cut}}{\Theta[(\lambda x.\Gamma[x] \bullet \Delta)] \vdash C}\lambda^{\downarrow}$$

Each of the proof transformations described above reduces either the depth of a use of Cut or the number of connectives in the formula that Cut targets. By repeatedly applying

them, each use of Cut will therefore be permuted and simplified until its left premise becomes an axiom, at which point the Cut can be eliminated entirely. ∎

Theorem 5.2. Every abstraction-free theorem of QRT has a Cut- and $\lambda^{\downarrow}$-free proof

Proof: Take a Cut-free proof of an abstraction-free theorem, and consider a $\lambda^{\downarrow}$-inference of maximal depth. Because the proof's conclusion is abstraction-free, there must be a lower $\lambda^{\uparrow}$-inference eliminating the abstraction introduced by $\lambda^{\downarrow}$. If that $\lambda^{\uparrow}$-inference directly follows, then the application of $\lambda^{\downarrow}$ is eliminable:

$$\dfrac{\dfrac{\Gamma[\Delta[\Theta]] \vdash A}{\Gamma[\Theta \bullet \lambda x.\Delta[x]] \vdash A}\,\lambda^{\downarrow}}{\Gamma[\Delta[\Theta]] \vdash A}\,\lambda^{\uparrow} \quad \rightsquigarrow \quad \Gamma[\Delta[\Theta]] \vdash A$$

Otherwise we can permute the $\lambda^{\downarrow}$-inference downward across any occurrence of →L, → R, and $\lambda^{\uparrow}$ until it meets up with a suitable $\lambda^{\uparrow}$-inference and can be removed. Here, for instance, is the way to permute a $\lambda^{\downarrow}$-inference that feeds into the left premise of an →L-inference:

$$\dfrac{\dfrac{\Gamma[\Delta[\Theta]] \vdash A}{\Gamma[\Theta \bullet \lambda x.\Delta[x]] \vdash A}\,\lambda^{\downarrow} \qquad \Lambda[B] \vdash C}{\Lambda[\Gamma[\Theta \bullet \lambda x.\Delta[x]] \bullet AB] \vdash C}\,{\to}\mathrm{L} \quad \rightsquigarrow \quad \dfrac{\dfrac{\Gamma[\Delta[\Theta]] \vdash A \qquad \Lambda[B] \vdash C}{\Lambda[\Gamma[\Delta[\Theta]] \bullet AB] \vdash C}\,{\to}\mathrm{L}}{\Lambda[\Gamma[\Theta \bullet \lambda x.\Delta[x]] \bullet AB] \vdash C}\,\lambda^{\downarrow}$$

Similar permutations are possible for all the other inference rules. The requirement that $\lambda^{\uparrow}$ and $\lambda^{\downarrow}$ not target abstractions will guarantee that it is possible to permute $\lambda^{\downarrow}$ across $\lambda^{\uparrow}$.

By repeating this procedure, we can eliminate each use of $\lambda^{\downarrow}$ one by one. Since the procedure doesn't introduce any new applications of Cut, the end result is a Cut- and $\lambda^{\downarrow}$ -free proof. ∎

Theorem 5.3. QRT provability is in NP.

Proof: By Theorem 5.2, every abstraction-free theorem of QRT has a proof using only the rules Ax, →L, →R, and $\lambda^{\uparrow}$. Read bottom to top, each application of →L and →R eliminates an occurrence of '→', and no rule introduces new ones. The number of uses of each of these rules in a proof is therefore bounded by the number of '→'s in the conclusion. Since each $\lambda^{\uparrow}$-inference introduces a λ-abstraction that must be removed by a corresponding application of →R, it can only be used as many times as →R is. Every abstraction-free theorem of QRT therefore has a proof whose size is polynomial in the theorem's length and which can serve as an efficient witness of theoremhood. ∎

The introduction to this section mentioned that LF repair has the same time complexity as QRT provability. The target result of this section is therefore established:

Theorem 5.4. LF repair is in NP.

6. LF repair is NP-Hard

6.1. Directed Graphs and Hamiltonian Cycles

The proof in this section uses some terminology from graph theory. As a reminder, a directed graph (digraph, for short) is a pair $\langle V, E\rangle$, where V is a finite set of *vertices* and $E \subseteq V \times V$ a set of *edges*. The first component of an edge is its *tail*, the second its *head*.

Given a vertex v, its *indegree* $\deg^-(v)$ and *outdegree* $\deg^+(v)$ are the number of edges of which v is a head and tail, respectively.

A *Hamiltonian cycle* in a digraph is a sequence of edges $e_1, e_2, ..., e_n$ such that for $0 < i < n$, the head of e_i is the tail of e_{i+1}, the head of e_n is the tail of e_1, and every vertex occurs exactly once as a head and a tail of an edge in the sequence. The following decision problem is NP-complete (Garey and Johnson 1979):

DIRECTED HAMILTONIAN CYCLE
Given a digraph, does it have a Hamiltonian cycle?

6.2. Encoding Hamiltonian Cycles in Logical Forms

This section describes a polynomial-time reduction from DIRECTED HAMILTONIAN CYCLE to LF REPAIR. Let $G = \langle V, E \rangle$ be a digraph with $n > 0$ vertices and $m \geq n$ edges. It's convenient to assume that the vertices of G are an initial segment of the natural numbers, that is, $V = \{1, ..., n\}$. I will now construct an instance of LF REPAIR whose answer is "yes" iff G has a Hamiltonian cycle. The method has some similarities to the one in Krantz and Mogbil (2001), who encode DIRECTED HAMILTONIAN CYCLE in the multiplicative fragment of linear logic.

The set of atomic types for the instance consists of a type v for each $v \in V$, plus the additional types a, b, c, d, e, and t. That is,

$$\mathbb{A} = \{1, ..., n\} \cup \{\mathsf{a}, \mathsf{b}, \mathsf{c}, \mathsf{d}, \mathsf{e}, \mathsf{t}\}. \tag{4}$$

When describing the reduction below, I'll use the following abbreviations for types:

$$\begin{aligned} E_u^v &\coloneqq (\mathsf{a} \to u) \to \mathsf{b} \to v, \\ V_v &\coloneqq (\mathsf{a} \to v) \to \mathsf{c} \to v, \\ G_v &\coloneqq (\mathsf{a} \to \mathsf{t}) \to \mathsf{d} \to v, \\ G^v &\coloneqq (\mathsf{a} \to v) \to \mathsf{e} \to \mathsf{t}. \end{aligned} \tag{5}$$

The rest of this subsection shows how to build an LF Γ such that $\Gamma \to^*_{\mathrm{QR}} \Delta$ for some Δ of type t if and only if G has a Hamiltonian cycle. Without loss of generality, I assume the cycle begins at the vertex 1. To make the construction more concrete, I will illustrate it as I go using the digraph in Figure 10. Notice that the graph has a Hamiltonian cycle: $(1, 3), (3, 2), (2, 4), (4, 1)$.

I'll build Γ up in four layers, defining an LF Γ_1 and three contexts Γ_2, Γ_3, Γ_4, such that

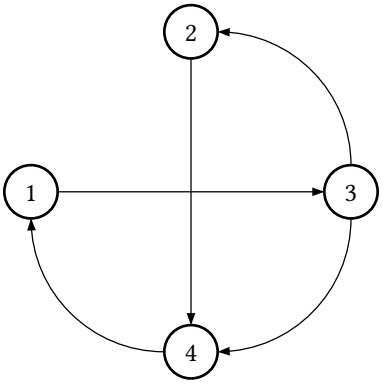


Figure 10: A directed graph.

$$\Gamma = \Gamma_4[\Gamma_3[\Gamma_2[\Gamma_1]]]. \tag{6}$$

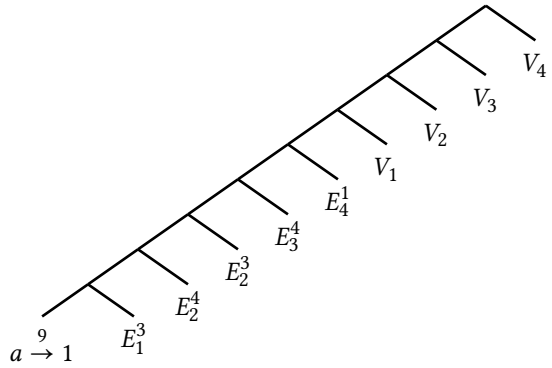
The plan is as follows. Γ_1 will contain a leaf E_u^v for each edge (u, v) in G and a leaf V_v for each vertex v in G. In order to have a chance at making Γ well typed, each one of these leaves will need to undergo QR. The context Γ_2 will have n positions to which n of the leaves E_u^v from Γ_1 can undergo QR, as well as n positions for the V_v. By design, the types will properly compose if and only if the n corresponding edges in G form a Hamiltonian cycle.

The $m - n$ leaves E_u^v corresponding to the edges not used in the cycle will also need a place to which they can QR. The context Γ_4 will provide $m - n$ positions for them. Since a Hamiltonian cycle enters and exits every vertex exactly once, for each vertex v there are $\deg^-(v) - 1$ edges with head v and $\deg^+(v) - 1$ edges with tail v that do not occur in the cycle. Γ_3 will accordingly contain $\deg^+(v) - 1$ leaves G_v and $\deg^-(v) - 1$ leaves G^v. For each E_u^v that QRs into Γ_4, G_u and G^v will QR right below and above it in Γ_4. This last step is needed purely to get the types to combine properly.

Now for the technical details. Let $(u_1, v_1), ..., (u_m, v_m)$ be G's edges, and let

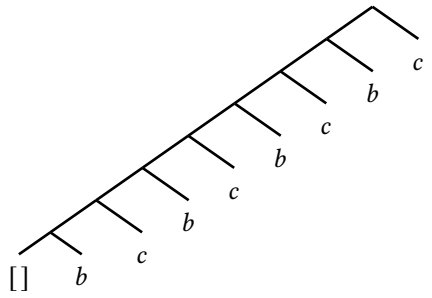
$$\Gamma_1 = \left(\mathrm{a} \xrightarrow{m+n} 1\right) \bullet E_{u_1}^{v_1} \bullet \cdots \bullet E_{u_m}^{v_m} \bullet V_1 \bullet \cdots \bullet V_n. \tag{7}$$

In the case of the digraph in Figure 10, Γ_1 looks as follows:



The intention is that every E_u^v and V_v will take scope and leave a trace of type a. The leaf of type $\mathrm{a} \xrightarrow{m+n} 1$ will consume these traces one by one, producing a subtree of type 1.

Let $\Delta = [\,] \bullet \mathrm{b} \bullet \mathrm{c}$ and let $\Gamma_2 = \Delta^n$. For the digraph in Figure 10, $\Gamma_2 = \Delta^4$, which looks like



If E_u^v and V_v each QR directly below the occurrences of b and c respectively in $\Delta[u]$, then the resulting subtree will have type v. Δ is therefore a gadget for simulating the traversal of an edge from u to v. The left side of Figure 11 illustrates the simulation with E_1^3 and V_3.

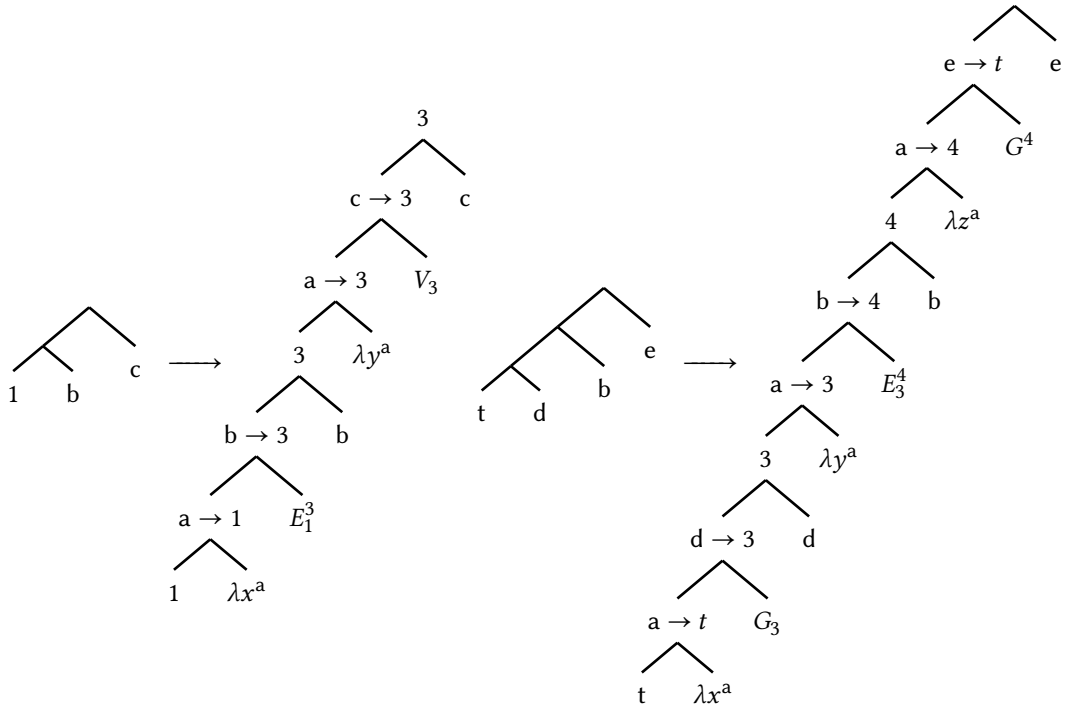


Figure 11: Simulating the traversal of an edge (left) and the disposal of an unused edge (right). Interior nodes are annotated with the type of the subtree they dominate. λ-abstractions are now shown on the right, just for cosmetics.

Just for intuition, one can imagine that QR of E_u^v takes us from the state "exiting u" to the state "entering v", and QR of V_v leads from the state "entering v" to the state "exiting v."

By nesting n copies of Δ, Γ_2 can simulate a path of length n in G. Notice that simulating the traversal of an edge with head v requires raising an occurrence of V_v. Since Γ will contain exactly one leaf V_v for each vertex v, the paths Γ_2 can simulate are those that visit each vertex exactly once, that is, Hamiltonian paths.

Γ_3 is the most complicated. First define $a \dot{-} b = \max(a - b, 0)$. Now for each $v \in V$ let

$$\Phi_v = [\,] \bullet \underbrace{G_v \bullet \cdots \bullet G_v}_{\deg^+(v) \dot{-} 1 \text{ times}}, \quad \Psi_v = [\,] \bullet \underbrace{G^v \bullet \cdots \bullet G^v}_{\deg^-(v) \dot{-} 1 \text{ times}} \tag{8}$$

and let

$$\Phi = \Phi_1[\Phi_2[\ldots\Phi_n]] \quad \text{and} \quad \Psi = \Psi_1[\Psi_2[\ldots\Psi_n]]. \tag{9}$$

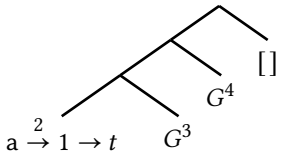
Also let

$$k = \sum_{v=1}^{n} \big(\deg^+(v) \dot{-} 1\big) + (\deg^-(v) \dot{-} 1). \tag{10}$$

Then

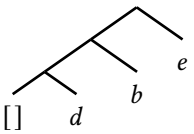
$$\Gamma_3 = \Psi\left[\Phi\left[\mathrm{a} \xrightarrow{k} 1 \to \mathrm{t}\right]\right] \bullet [\,]. \tag{11}$$

In the case of the digraph in Figure 10 (where only Φ_3 and Ψ_4 are nontrivial), Γ_3 would be:



To make Γ well typed, each G_u and G^v will need to undergo QR and leave behind a trace of type a. The leaf of type $\mathsf{a} \xrightarrow{k} 1 \to \mathsf{t}$ will consume these traces, resulting in a subtree of type $1 \to \mathsf{t}$.

Finally, let $\Sigma = [\,] \bullet \mathsf{d} \bullet \mathsf{b} \bullet \mathsf{e}$, and let $\Gamma_4 = \Sigma^{m-n}$. In the case of the digraph in Figure 10, Γ_4 would be:



The context Σ is a gadget that works more or less like Δ. If G_u, E_u^v, and G^v each QR directly below the occurrences of d, b, and e respectively in $\Sigma[\mathsf{t}]$, the resulting subtree will have type t. The right side of Figure 11 illustrates the procedure with G_3, E_3^4, and G^4. Together $\Sigma[\mathsf{t}]$, G_u, and G^v work as a kind of disposal mechanism for the leaf E_u^v, providing it a place to which it can QR and returning a subtree of type t.

Γ_4 consists of $m - n$ nested copies of Σ. When G has a Hamiltonian cycle, Γ_1 will contain $m - n$ leaves E_u^v coming from edges not used in the cycle. Γ_4's purpose is to act as a waste bin of sorts for these unused edges. Notice that "disposing" of a leaf E_u^v requires raising both a leaf G_u and a leaf G^v from Γ_3. As mentioned earlier, for each vertex v there are $\deg^-(v) - 1$ edges with head v and $\deg^+(v) - 1$ edges with tail v that do not occur in the cycle. Γ_3 therefore provides exactly the right number of leaves G_u and G^v to dispose of all the unused edges.

That completes the encoding. Notice that $\Gamma = \Gamma_4[\Gamma_3[\Gamma_2[\Gamma_1]]]$ will always be linear in the size of the input, and can be constructed in polynomial time.

Figure 12 shows the end result of applying the procedure to the digraph in Figure 10. I encourage the reader to use the Hamiltonian cycle in this digraph to work out where the different leaves E_u^v and V_v must raise in order to create a well-typed structure.

6.3. Correctness

It remains to show that the reduction is correct, that is, that G has a Hamiltonian cycle if and only if the answer to the LF REPAIR-instance constructed in the previous subsection is "yes".

First the easy direction.

Theorem 6.1. If G has a Hamiltonian cycle, then there exists an LF Δ of type t such that $\Gamma \to_{\mathrm{QR}}^* \Delta$.

Proof: Let

$$(u_1, u_2), (u_2, u_3), \ldots, (u_{n-1}, u_n), (u_n, u_1) \tag{12}$$

be a Hamiltonian cycle in G. Without loss of generality, assume that $u_1 = 1$. We can derive an appropriate Δ of type t from Γ as follows. Note that every QR step in the derivation below will leave a trace of type a.

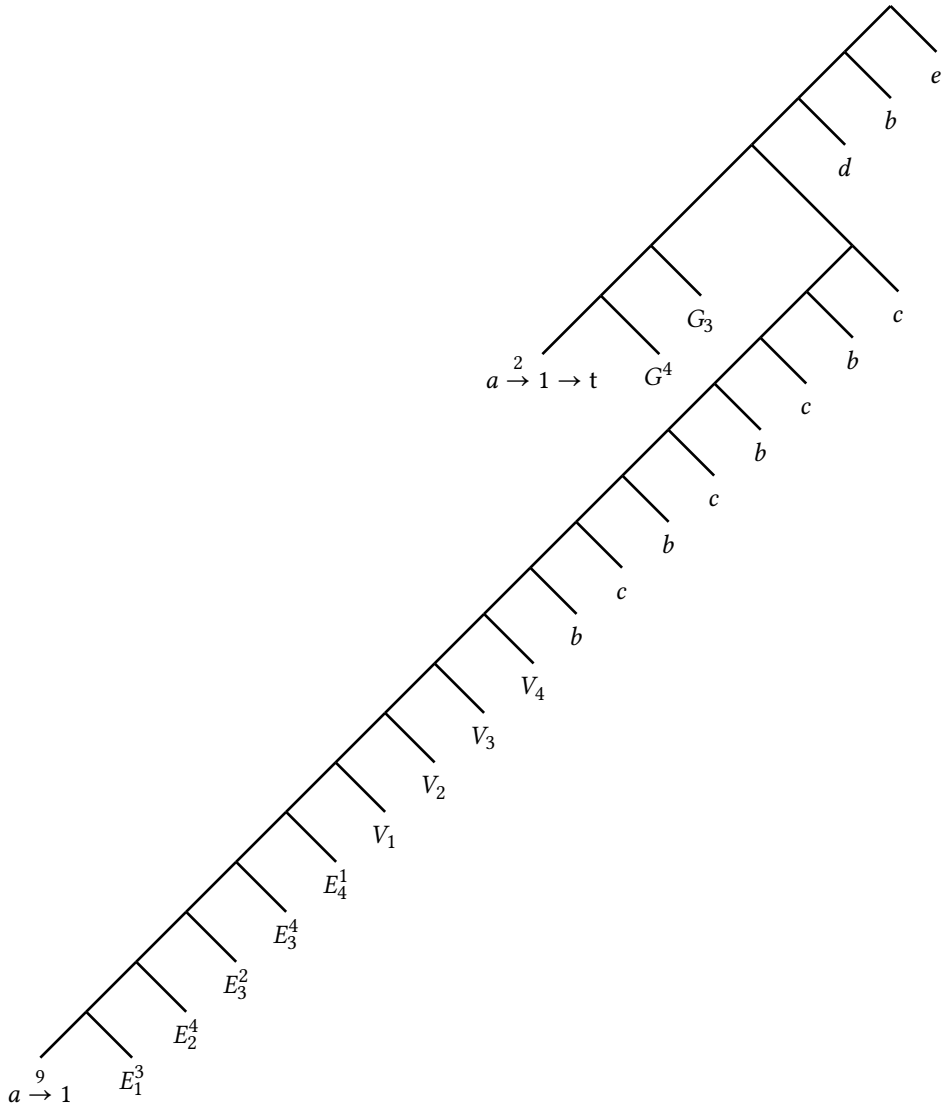


Figure 12: The LF REPAIR-instance for the digraph shown in Figure 10.

First QR $E_{u_1}^{u_2}$ and V_{u_2} directly below the lowest occurrence of b and c respectively in Γ_2 (compare the left side of Figure 11). Then QR $E_{u_2}^{u_3}$ and V_{u_3} directly below the *second*-lowest b and c respectively in Γ_2. Continue this process for the rest of the cycle, finishing with $E_{u_n}^{u_1}$ and V_{u_1}.

Exactly $m - n$ leaves of the form E_u^v in Γ_1 are still waiting to QR, each one of them corresponding to an edge not used in the Hamiltonian cycle. For each vertex v, exactly $\deg^-(v) - 1$ of these leaves correspond to an edge with head v and $\deg^+(v) - 1$ to an edge with tail v.

For each b in Γ_4, QR one of the $m - n$ remaining leaves E_u^v's directly below it. Then QR an instance of G_u and G^v from Γ_3 directly below the adjacent occurrences of d and e, respectively (compare the right side of Figure 11). Since the number of leaves G_u and G^v

is $\deg^+(v) - 1$ and $\deg^-(u) - 1$ respectively, by the end of this process every G_u and G^v from Γ_3 will have undergone QR.

The LF resulting from these $3m - n$ applications of QR is the Δ we wanted. An easy check shows that it has type t. ∎

Now for the harder direction, which comes down to showing that the procedure in the previous proof is the only way to make Γ well typed.

Theorem 6.2. Suppose that $\Gamma \to^*_{\text{QR}} \Delta$ for some Δ of type t. Then G has a Hamiltonian cycle.

Proof: By Lemma 4.1, we can assume that Δ is normalized. We now make six claims about Δ. Below, I refer to a leaf that occurs in Γ as a "Γ-leaf."

(i) Every trace in Δ is of order one and therefore of atomic type. The claim follows from Lemma 4.2 and the fact that all of Γ's leaves have order at most three. A consequence is that nothing in the derivation $\Gamma \to^*_{\text{QR}} \Delta$ undergoes QR more than once, since this would leave a trace of atomic type next to an abstraction, contradicting the fact that Δ is normalized.

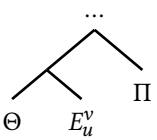
(ii) Every internal node of Δ has a type of order at most two. To prove the claim, suppose it were false and consider a maximally deep internal node of order greater than two. By (i), all abstractions are over variables of order one, so the node is not an abstraction (otherwise there would be a deeper internal node of the same order). It therefore has the form $\Theta \bullet \Lambda$, where Θ has type $\alpha\beta$, Λ has type α, and $\text{ord}(\beta) > 2$. Since $\text{ord}(\tau(\Theta)) = \text{ord}(\alpha\beta) > 2$, Θ cannot be an internal node, so it must be a leaf. But each leaf of Δ is either a trace of order one or a Γ-leaf, and by inspection no Γ-leaf has the form $\alpha\beta$ where $\text{ord}(\beta) > 2$. So no such internal node can exist.

(iii) Δ contains no "double abstractions," i.e., it is impossible to write $\Delta = \Theta[\lambda x\,(\lambda y\,\Lambda)]$. Suppose otherwise, and consider a "double abstraction" of *minimal* depth, so that we can write Δ as $\Theta\left[\Pi \bullet \lambda x^\alpha\,\left(\lambda y^\beta\,\Lambda\right)\right]$. Because Δ is normalized, Π must have a type of the form $(\alpha\beta\gamma)\delta$. This means that Π's type has order at least three, and so by (ii) Π must be a leaf. But no leaf of Δ has the form $(\alpha\beta\gamma)\delta$, a contradiction.

(iv) If an abstraction $\lambda x^\alpha\,\Pi$ occurs in Δ, then $\Delta = \Theta[l \bullet \lambda x^\alpha\,\Pi]$ for some context Θ and leaf l of type E^v_u, V_v, G_v, or G^v. This follows from (i), (ii), (iii), and the fact that Δ is normalized. A consequence is that in a derivation $\Gamma \to^*_{\text{QR}} \Delta$, the only elements that undergo QR are leaves of type E^v_u, V_v, G_v, or G^v.

(v) Every trace in Δ has type a. If x^α is a trace in Δ, then it must be bound by some abstraction, so by (iv) we can write Δ as $\Theta[l \bullet \lambda x^\alpha\,\Pi[x^\alpha]]$, where l is a Γ-leaf. Since Δ is normalized, $l = (\alpha\beta)\gamma$ for some types β, γ. Inspecting the leaves of Γ shows that α can only be a.

(vi) Call the sister of the mother of a node in a tree its *aunt*. Then the aunt of each leaf E^v_u, V_v, G_v, and G^v in Δ is a Γ-leaf of type b, c, d or e, respectively. Consider, for example, a leaf E^v_u occurring in Δ with sister Θ and aunt Π, as shown below:



Θ would need order at least 4 to take

$$E^v_u \coloneqq (\text{a} \to u) \to \text{b} \to v \tag{13}$$

as an argument, but no node in Δ has order higher than 3. So Θ must have type $a \to u$ and $\Theta \bullet E^v_u$ must have type $b \to v$. For Π to take $b \to v$ as an argument, it would need to have order at least 3. The only such elements of Δ are the Γ-leaves, but no Γ-leaf has a type of the form $(b \to v) \to \gamma$. So Π must have type b. Since all the traces in Δ have type a, the only nodes of type b in Δ are the m Γ-leaves coming from Γ_2 and Γ_4, and therefore Π is a Γ-leaf of type b, as we wanted to show. Similar arguments apply to the leaves of type V_v, G_v, and G^v. Notice that none of the leaves E^v_u, V_v, G_v, and G^v has an aunt of type b, c, d, or e in their base positions in Γ, so they must have all undergone QR in the derivation $\Gamma \to^*_{\text{QR}} \Delta$.

The upshot of all these observations is that in the derivation $\Gamma \to^*_{\text{QR}} \Delta$, every leaf of type E^v_u, V_v, G_v, and G^v undergoes QR *exactly once* (by (I) and (VI)); lands in a position directly below a unique leaf labeled b, c, d, or e respectively (by (VI)); leaves a trace of type a (by (V)); and *nothing else* undergoes QR (by (IV)).

We can now extract a Hamiltonian cycle from Δ. Exactly $2n$ leaves E^v_u and V_v in Δ underwent QR into Γ_2. Order them by decreasing depth:

$$E^{v_1}_{u_1}, V_{w_1}, E^{v_2}_{u_2}, V_{w_2}, \ldots, E^{v_n}_{u_n}, V_{w_n}. \tag{14}$$

In order for Δ to be well-typed, it must be the case that $u_1 = v_n = w_n = 1$ and

$$v_i = w_i = u_{i+1}, \quad 1 \leq i < n. \tag{15}$$

So we can rewrite the sequence above as

$$E^{u_2}_{u_1}, V_{u_2}, E^{u_3}_{u_2}, V_{u_3}, \ldots, E^{u_1}_{u_n}, V_{u_1}. \tag{16}$$

Since $V_{u_1}, \ldots, V_{u_n}$ are pairwise distinct, so are $u_1, \ldots, u_n$, and

$$(u_1, u_2), (u_2, u_3), \ldots, (u_n, u_1) \tag{17}$$

is therefore a Hamiltonian cycle in G. ∎

Putting Theorem 5.4, Theorem 6.1, and Theorem 6.2 together proves the main claim of this paper:

Theorem 6.3. LF REPAIR is NP-complete.

Section 5 mentioned that LF REPAIR has the same time complexity as provability in QRT. So a corollary of the above is:

Theorem 6.4. QRT PROVABILITY is NP-complete.

7. Discussion

A prominent semantic tradition assumes that a level of representation called Logical Form mediates between a sentence's surface syntax and its meaning. The most basic property of an LF is that it is completely free of type clashes and is therefore a suitable input to a compositional semantics. This paper set out to answer a fundamental but unanswered question about this tradition: given a surface structure, can a well-typed LF be computed efficiently? The preceding sections formalized this question as a decision problem called LF REPAIR and proved that it is NP-complete. To make a concrete comparison, this result means that deciding whether a surface form has a well-typed LF is as complex as the Boolean satisfiability problem: to determine whether a formula built out of variables and Boolean connectives is true under some assignment of truth values to those variables.

If we make the plausible assumption that P ≠ NP, the result entails more generally that there is no efficient algorithm for mapping a surface form onto a well-typed Logical Form.

What is the significance of this intractability result? In practical terms, it means that the task of finding a well-typed LF is not efficiently automatable. There may exist heuristic algorithms with acceptable performance in practice, just like SAT-solvers can determine, often very quickly, whether a Boolean formula is true under some valuation of its variables. But NP-completeness guarantees that any heuristic will fail on certain inputs (unless P = NP). If one wants an algorithm that never fails, one might not be able to improve significantly on exhaustive search. As useful as it would be to quickly find all the truth-conditionally distinct LFs for an arbitrary surface form, the task just isn't computationally feasible.

The theoretical significance of the NP-completeness proof will depend on one's prior beliefs about the relation between cognition and tractability. It's sometimes suggested that NP-completeness is exactly the level of complexity one should expect from the computations underlying language production and comprehension, and that any theory with lower complexity will fail to account for sufficiently complex linguistic phenomena (Ristad 1993). To someone who agrees with that claim, the results in this paper would suggest that QR strikes just the right balance: it is computationally neither too complex nor too simple.

But many philosophers and cognitive scientists have argued that NP-complete problems are too cognitively demanding and have therefore defended various kinds of *tractable cognition thesis* (Rooij et al. 2019). One version is the P-Cognition thesis, which claims that human cognition is limited to polynomial-time computations. Someone who believes that thesis might conclude from Section 6 that QR and LF, useful as they have been, require more computational power than the human brain can supply, and that they must be replaced by a new theory of scope. Alternatively, they might decide that LF REPAIR is too broad as a formalization of the computational task that the human mind performs during language comprehension; the mind, they think, is solving some suitably restricted version of LF REPAIR that is computable in polynomial time. If so, it is necessary to state what exactly that restricted version of the problem is. Whatever option they choose, the believer in the P-Cognition thesis has work to do, either in developing a computationally tractable theory of scope or in stating a restricted but still linguistically relevant version of LF REPAIR.

Rooij et al. (2019) argue for a loosening of the P-Cognition thesis called the FPT-cognition thesis, which claims that human cognition is limited to *fixed-parameter tractable* (FPT) computations. Informally, a problem is fixed-parameter tractable if it's solvable in polynomial time so long as one or more specific parameters of the problem are held fixed. Any supra-polynomial behavior of an FPT problem therefore comes from these parameters, and as long as they are sufficiently small, the problem remains tractable. For example, one parameter of the Boolean satisfiability problem is the number k of distinct variables allowed in a formula. If k is held fixed, then the problem becomes solvable in polynomial time: given a formula of size n, it's possible to check all possible valuations of its variables in $O\left(2^k \cdot n\right)$ time. Since this is polynomial in n, Boolean satisfiability is fixed-parameter tractable, even though it's NP-complete.

For Rooij et al., the FPT-cognition thesis is not just a restriction on the space of possible computational-level theories. It's a methodological exhortation to investigate what parameters of cognitively relevant computational decision problems are actually responsible

for intractability, and to consider whether those parameters are small enough in practice to guarantee real-world tractability. What happens if we look at LF REPAIR in this spirit?

One natural parameter to consider is the number of allowed atomic types. The reduction from the directed Hamiltonian cycle problem in Section 6 introduced distinct atomic types $1, \ldots, n$ for each of the n vertices of the input graph. Because the number of vertices in the input graph can be arbitrarily large, so is the number of atomic types introduced in the reduction. Since one parameter of LF REPAIR is the number k of allowed atomic types, one way to invalidate the reduction is therefore to hold k fixed. But it turns out that this restriction has no appreciable effect on complexity. It's possible to encode the types $1, \ldots, n$ from the reduction in "unary" using just one type v: represent 1 by v, 2 by $\mathsf{v} \to \mathsf{v}$, 3 by $\mathsf{v} \to \mathsf{v} \to \mathsf{v}$, etc. If each occurrence of $1, \ldots, n$ in Section 6 is replaced by its unary encoding, one ends up with a modified version of the reduction that uses only the atomic types a, b, c, d, e, t, and v. LF REPAIR is therefore intractable even for small inventories of atomic types.

What about other parameters? The preceding sections worked with a particularly powerful version of QR. It was allowed to raise practically any subtree to any position and to leave a trace of any type, so long as the final LF is well typed. It's therefore natural to wonder whether LF REPAIR would become tractable if some parameter of the operation QR itself were restricted. But many natural candidates won't make a difference. For example, one might constrain the complexity of the traces that QR creates by requiring that their order be below some threshold k. Setting k to 1 would force QR to only leave traces of atomic type (Poole 2024). It turns out, however, that LF REPAIR is intractable even if k is fixed to 1. The reason is that the reduction in Section 6 only depended on leaving traces of the atomic type a anyway. More precisely, the LF constructed by the reduction is repairable using QR if and only if it's repairable using QR restricted to leaving traces of a single atomic type. The intractability of LF REPAIR is therefore independent of constraints on the order of traces.

There are other natural constraints on QR worth considering. One might, for example, ban "cyclic QR", i.e., require that no subtree undergo QR more than once. Or one might ban parasitic scope, where an application of QR wedges a constituent between the raised expression and the λ-abstractor introduced by a previous application of QR (Barker 2007). But just like the reduction in Section 6 made no use of higher-order traces, it made no use of cyclic QR or parasitic scope either. So LF REPAIR would be intractable even if it were reformulated to disallow cyclic QR or parasitic scope.

One constraint that could lead to a more tractable decision problem is to require that any expression targeted by QR move to the top of the tree. This requirement would obviously invalidate the reduction in Section 6, which depends on the possibility of raising expressions to intermediate positions. If moreover all scope-takers have types of the form $(\alpha \to \beta) \to \gamma$ where β and γ are atomic, then linking them up at the top of a tree should be computationally easy. We can conceive of each scope-taker as an edge of a directed graph linking nodes labeled β and γ, and then the existence of a well-typed LF basically reduces to the tractable problem of determining whether a certain graph has an Eulerian path. This observation highlights that the core computational challenge with the LF in Section 6 is deciding which scope-takers will raise into the middle region of the LF and which will raise into the upper region. Once the partition is complete, finding a well-typed arrangement of the scope-takers is relatively easy.

It's an open question what linguistically interesting reformulations of LF REPAIR could make the problem tractable when they are held fixed. This preliminary discussion has

established that some obvious candidates don't work and proposed one candidate that should, but there remain plenty of possibilities to consider. For anyone who is committed both to QR and to the FPT-cognition thesis, it will be important to keep exploring the space of parameters.

Even though LF REPAIR is worst-case intractable, the concrete instances that semanticists have to solve are by and large easy. This discrepancy lends some plausibility to the idea that the LFs that semanticists work with are subject to implicit constraints that reduce complexity, and that there is a restricted version of LF REPAIR that is relevant to natural language. It's difficult to say with mathematical precision what these implicit constraints are. But it seems relevant that scope-takers tend in practice to have types of the form $(\alpha \to \beta) \to \beta$, meaning that their "input" and "output" type are each β.[5] The scope-takers in Section 6, by contrast, had types of the form $(\alpha \to \beta) \to \gamma$ where $\beta \neq \gamma$. The difference in input and output type made it possible to use the semantic types to keep track of state (which vertex of a graph one is currently visiting) and to use the scope-takers to perform state transitions. The result was a game of dominoes where input and output types must link together in an intricate chain in order to build a well-typed LF. Intuitively, the freedom to introduce types of the form $(\alpha \to \beta) \to \gamma$ with $\beta \neq \gamma$ for arbitrarily many β and γ seems necessary for NP-completeness. At the very least, the reduction strategy in Section 6 makes essential use of this freedom.

Before closing, it's worth briefly mentioning some possible extensions of the results of this paper. It is likely that a reduction strategy similar to the one in Section 6 could establish NP-completeness results for other models of scope-taking. The type-logical grammar NL_λ, for example, is a close relative of the formalism QRT introduced in Section 5, and is likely to have the same time complexity (Barker 2019; Moot 2025). Continuized CCGs are another relative that probably have an NP-complete parsing problem (Barker and Shan 2014; White et al. 2017). Establishing these extensions would bolster the point that this paper's results are not about the operation QR in itself, but about a broad class of scope-taking formalisms of which QR is just one prominent exemplar.

5. There are exceptions to this dependency. In some theories, for example, an expression's semantic type records whether it contains any unbound anaphors (Barker and Shan 2014; Bumford and Charlow, n.d.). Suppose that in such a theory $\mathrm{e} \Rightarrow \mathrm{t}$ is the type of a proposition containing a single free pronoun of type e. If an occurrence of a quantifier like *everyone* binds a pronoun in its scope, then that occurrence should have the type $(\mathrm{e} \to (\mathrm{e} \Rightarrow \mathrm{t})) \to \mathrm{t}$. In this case, the input type $\mathrm{e} \Rightarrow \mathrm{t}$ differs from the output type t.

A. Footnote 1 in Barker (2020)

The definition of Quantifier Raising in Section 4 disallows raising abstractions. The definition of the logic QRT in Section 5 places a parallel restriction on the structural rules $\lambda^{\uparrow}$ and $\lambda^{\downarrow}$. Neither restriction is present in the definitions of QR and QRT in Barker (2020). This appendix explains why the definitions in this paper are different from Barker's, and offers some more justification for the ban against raising abstractions.

In a footnote to his proof that QRT is decidable, Barker in fact acknowledges that raising abstractions is a problem and suggests a solution. (Note that $\text{QR}_{\uparrow}$ is Barker's name for what I call $\lambda^{\uparrow}$.)

> After publication of the early-access version of this paper, I discovered that this proof is incomplete due to the way that commutativity interacts with remnant movement (i.e., when $\text{QR}_{\uparrow}$ raises an abstraction structure). Rather than adding the details of the complete proof here, I will suggest repairing the proof by adopting two adjustments. The first adjustment is to replace unordered type structures … with ordered pairs. This would be a change in the interpretation of the structural connective •, with no change in the appearance of any type structure. The second adjustment is to add inference rules to QRT … for a new logical connective that allows the argument of an implication to appear on its right hand side. Having both left and right implications is standard in Type Logical Grammar, where the connectives are written as $A\backslash B$ and B/A. Logical proofs would remain unchanged, except that inferences that rely on structures being unordered … will be replaced with /L inferences, without any change to the premises or the conclusions. With these adjustments in place, the decidability argument given here is correct and complete as written.

I will try to spell out the issue that the footnote alludes to and what the suggested remedy might look like.

Section 5 already introduced the problem that Barker noticed in "the way that commutativity interacts with remnant movement." The following schematic derivation displays the issue clearly:

$$\dfrac{\dfrac{\dfrac{\dfrac{\Gamma[\Delta[A \bullet B]] \vdash C}{\Gamma[A \bullet \lambda x.\Delta[x \bullet B] \vdash C]}\,\lambda^{\downarrow}}{(\lambda x.\Delta[x \bullet B]) \bullet \lambda y.\Gamma[A \bullet y] \vdash C}\,\lambda^{\downarrow}}{\Delta[(\lambda y.\Gamma[A \bullet y]) \bullet B] \vdash C}\,\lambda^{\uparrow}}{\Delta[\Gamma[A \bullet B]] \vdash C}\,\lambda^{\uparrow}$$

In essence, this small proof shows that without appropriate constraints, QRT implicitly obeys a bizarre structural rule that equates all LFs of the form $\Gamma[\Delta[A \bullet B]]$ and $\Delta[\Gamma[A \bullet B]]$. This fact has the undesirable consequence that QRT is free to all but entirely ignore the hierarchical structure of the LFs to the left of the turnstile.

The first step of Barker's two-pronged solution is to relinquish the commutativity of '•', or, put differently, to treat LFs as *ordered* rather than *unordered* trees. An immediate good consequence of this change is that it invalidates the derivation above. Why? If '•' is non-commutative, then the rule $\lambda^{\uparrow}$ only licenses elimination of a λ-abstraction when the argument is to the left. The higher application of $\lambda^{\uparrow}$ above, where the argument is to the right, is therefore illicit.

But the switch to ordered trees also reduces the expressive power of the logic dramatically. Because the inference rules now require that *all* arguments appear to the left, even simple sequents like et • e ⊢ t have become unprovable. The second step of Barker's solution is intended to remedy this deficit. Following Lambek (1958), he suggests replacing the ambidirectional connective '→' with the two new unidirectional connectives '/' and '\' and adding appropriate inference rules. In the resulting logic, the analogue of a sequent like et • e ⊢ t would be e/t • e ⊢ t

What should the inference rules for these two new connectives be? Each of them requires a left and a right introduction rule. It is clear what the left introduction rules should be:

$$\frac{\Gamma \vdash A \qquad \Delta[B] \vdash C}{\Delta[\Gamma \bullet A\backslash B] \vdash C}\backslash\mathrm{L} \qquad\qquad \frac{\Gamma \vdash A \qquad \Delta[B] \vdash C}{\Delta[B/A \bullet \Gamma] \vdash C}/\mathrm{L}$$

There are multiple choices for \R and /R that one might consider, but a sensible one is to adopt the same rules as the Lambek calculus. (If one makes this choice, it will not, however, be the case that old QRT proofs carry over "without any change to the premises or the conclusions," so Barker may have had different rules in mind.)

$$\frac{A \bullet \Gamma \vdash B}{\Gamma \vdash A\backslash B}\backslash\mathrm{R} \qquad\qquad \frac{\Gamma \bullet A \vdash B}{\Gamma \vdash B/A}/\mathrm{R}$$

With these definitions, a variant of the former rule →R is derivable in two steps:

$$\dfrac{\dfrac{\Gamma[A] \vdash B}{A \bullet \lambda x.\Gamma[x] \vdash B}\lambda^{\downarrow}}{\lambda x.\Gamma[x] \vdash A\backslash B}\backslash\mathrm{R}$$

The resulting logic is just the product-free, non-associative Lambek calculus enriched with the two rules $\lambda^{\uparrow}$ and $\lambda^{\downarrow}$. Equivalently, it is the fragment of the multi-modal type-logical grammar NL_{λ} that remains after dropping all but the "continuation mode" connectives (Barker 2019; Moot 2025).

The above solution seems workable. So why, rather than use it, does this paper take the approach of defining QRT with a blanket prohibition against raising abstractions? One reason is convenience: the version of QRT that I use has extremely few rules and connectives, which simplifies both the presentation and the proofs. Splitting '→' into '/' and '\' would make everything busier. The other reason is a desire for QRT and the type theory it's built on to closely track the ways that linguists think about Logical Form and Quantifier Raising. Typically, rules of semantic combination are indifferent to linear order, and no distinction is drawn at the level of semantic types between functions that require arguments on their left and on their right. The presentation of QRT in Section 5 respects these conventions.

References

Barker, Chris. 2007. "Parasitic Scope." *Linguistics and Philosophy* 30 (4): 407–44.

Barker, Chris. 2015. "Scope." In *The Handbook of Contemporary Semantic Theory*, edited by Shalom Lappin and Chris Fox. John Wiley & Sons.

Barker, Chris. 2019. "NL$_\lambda$ as the Logic of Scope and Movement." *Journal of Logic, Language and Information* 28 (2): 217–37.

Barker, Chris. 2020. "The Logic of Quantifier Raising." *Semantics and Pragmatics* 13: 1–40.

Barker, Chris, and Chung-chieh Shan. 2014. *Continuations and Natural Language*. Oxford University Press.

Bhatt, Rajesh, and Veneeta Dayal. 2007. "Rightward Scrambling as Rightward Remnant Movement." *Linguistic Inquiry* 38 (2): 287–301.

Bobaljik, Jonathan David. 2002. "A-Chains At The PF-Interface: Copies And 'Covert' Movement." *Natural Language & Linguistic Theory* 20 (2): 197–267.

Bumford, Dylan, and Simon Charlow. Pre-published. "Dynamic Semantics with Static Types."

Charlow, Simon. 2021. "Post-Suppositions and Semantic Theory." Pre-published.

Cooper, Robin. 1983. *Quantification and Syntactic Theory*. D. Reidel.

Fox, Danny. 1995. "Economy and Scope." *Natural Language Semantics* 3: 283–341.

Garey, Michael R., and David S. Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.

Heim, Irene. 1999. "Notes on Superlatives." Massachusetts Institute of Technology.

Heim, Irene. 2000. "Degree Operators and Scope." *Semantics and Linguistic Theory* 10: 40–64.

Heim, Irene, and Angelika Kratzer. 1998. *Semantics in Generative Grammar*. Blackwell Textbooks in Linguistics 13. Wiley Blackwell.

Hendriks, Herman. 1993. "Studied Flexibility." PhD thesis, University of Amsterdam.

Huang, C.-T. James. 1993. "Reconstruction and the Structure of VP: Some Theoretical Consequences." *Linguistic Inquiry* 24 (1): 103–38.

Iatridou, Sabine, and Hedde Zeijlstra. 2013. "Negation, Polarity, and Deontic Modals." *Linguistic Inquiry* 44 (4): 529–68.

Jacobson, Pauline. 1999. "Towards a Variable-Free Semantics." *Linguistics and Philosophy* 22 (2): 117–84.

Jacobson, Pauline. 2012. "Direct Compositionality." In *The Oxford Handbook of Compositionality*, edited by Wolfram Hinzen, Edouard Machery, and Markus Werning. Oxford University Press.

Kennedy, Chris. 1997. "Projecting the Adjective: The Syntax and Semantics of Gradability and Comparison." PhD thesis, University of California, Santa Cruz.

Kiselyov, Oleg, and Chung-chieh Shan. 2014. "Continuation Hierarchy and Quantifier Scope." In *Formal Approaches to Semantics and Pragmatics: Japanese and Beyond*, edited by Elin McCready, Katsuhiko Yabushita, and Kei Yoshimoto. Springer.

Kobele, Gregory M. 2010. "Inverse Linking via Function Composition." *Natural Language Semantics* 18 (2): 183–96.

Kobele, Gregory M. 2018. "The Cooper Storage Idiom." *Journal of Logic, Language and Information* 27 (2): 95–131.

Krantz, Thomas, and Virgile Mogbil. 2001. "Encoding Hamiltonian Circuits into Multiplicative Linear Logic." *Theoretical Computer Science* 266 (1–2): 987–96.

Kubota, Yusuke, and Robert Levine. 2019. "Modal Auxiliaries and Negation: A Type-Logical Account." In *Logic, Language, Information, and Computation*, edited by Rosalie Iemhoff, Michael Moortgat, and Ruy De Queiroz. Springer.

Lambek, Joachim. 1958. "The Mathematics of Sentence Structure." *The American Mathematical Monthly* 65 (3): 154–70.

Landman, Meredith Lizabeth. 2006. "Variables in Natural Language." PhD thesis, University of Massachusetts Amherst.

May, Robert. 1977. "The Grammar of Quantification." PhD thesis, Massachusetts Institute of Technology.

McCawley, James D. 1996. "The Focus and Scope of *Only*." In *Discourse and Meaning: Papers in Honor of Eva Hajičová*, edited by Barbara H. Partee and Petr Sgall. John Benjamins Publishing Company.

Montague, Richard. 1973. "The Proper Treatment of Quantification in Ordinary English." In *Approaches to Natural Language: Proceedings of the 1970 Stanford Workshop on Grammar and Semantics*, edited by K. J. J. Hintikka, J. M. E. Moravcsik, and P. Suppes. Springer.

Moot, Richard. 2025. "Proof-Theoretic Aspects of the Logic of Scope." *Journal of Logic, Language and Information* 34 (5): 463–514.

Poole, Ethan. 2024. "(Im)possible Traces." *Linguistic Inquiry* 55 (2): 287–326.

Poole, Ethan, and Stefan Keine. 2024. "Not All Reconstruction Effects Are Syntactic." *Natural Language & Linguistic Theory* 42 (4): 1677–725.

Ristad, Eric Sven. 1993. *The Language Complexity Game*. MIT Press.

Rooij, Iris van, Mark Blokpoel, Johan Kwisthout, and Todd Wareham. 2019. *Cognition and Intractability: A Guide to Classical and Parameterized Complexity Analysis*. Cambridge University Press.

Rullmann, Hotze. 1995. "Maximality in the Semantics of *Wh*-Constructions." PhD thesis, University of Massachusetts, Amherst.

Stateva, Penka. 1999. "In Defense of the Movement Theory of Superlatives." *Proceedings of ESCOL 99*.

Strachey, Christopher, and Christopher P. Wadsworth. 1974. *Continuations: A Mathematical Semantics for Handling Full Jumps*. Oxford University Computing Laboratory, Programming Research Group.

Szabolcsi, Anna. 1997. *Ways of Scope Taking*. Studies in Linguistics and Philosophy 65. Kluwer.

Szabolcsi, Anna. 2010. *Quantification*. Cambridge University Press.

White, Michael, Simon Charlow, Jordan Needle, and Dylan Bumford. 2017. "Parsing with Dynamic Continuized CCG." In *Proceedings of the 13th International Workshop on Tree Adjoining Grammars and Related Formalisms*, edited by Marco Kuhlmann and Tatjana Scheffler. Association for Computational Linguistics.